

A GENETIC PROGRAMMING APPROACH FOR ALLOCATING UNDERGRADUATE PROJECT TOPICS

BY

ALEXANDER SUNDAY AMOWE
140404050

**A RESEARCH PROJECT WORK SUBMITTED TO THE
DEPARTMENT OF COMPUTER SCIENCE, FACULTY OF
SCIENCE, ADEKUNLE AJASIN UNIVERSITY, AKUNGBA
AKOKO, ONDO STATE, NIGERIA.**

**IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR
THE AWARD OF BACHELOR OF SCIENCE (B.Sc.) DEGREE IN
COMPUTER SCIENCE.**

SUPERVISOR
DR. O.O. AJAYI

APRIL, 2019

ABSTRACT

This study addresses inefficiencies in the manual allocation of project topics to undergraduate students in higher education institutions, with a case study of Adekunle Ajasin University, which often result in mismatches between students' research interests and supervisors' expertise. Therefore, it proposes a Genetic Programming (GP)-based model to optimize the allotment inefficiency with respect to research domains and preference criteria.

The Student-to-Project Topic Allocation (SPA) problem was formulated as a constrained optimization model that incorporates supervisors' capacity constraints and expertise levels, as well as student preference matrices. Specifically, a soft computing approach, primarily Genetic Programming, was adopted, with genetic operators, such as selection, crossover, and mutation, to generate an optimal program for topic assignment.

The system was evaluated through unit, integration, and system testing. The results further indicate that the GP achieved an optimization performance of 85.86%, demonstrating a measurable improvement over traditional manual allocation methods. Furthermore, the findings suggest that the proposed approach provides a robust and efficient framework for achieving equitable and preference-aligned allocation of research topics to students in higher education contexts.

KEYWORDS: Genetic Programming; Student-to-Project Allocation (SPA); Undergraduate Project Topics; Soft Computing; Optimization Model; Agile Development.

CHAPTER ONE

GENERAL INTRODUCTION

1.1 Introduction

As part of undergraduate degree programmes, students are typically required to undertake a final-year research project. In most cases, available projects are advertised, and students develop preference rankings after reviewing project descriptions. Similarly, supervisors may express preferences regarding students or the projects they offer. Additionally, constraints such as limits on the number of students per project and supervisory capacity must be considered in the allocation process (David & Gregg, 2008).

Although undergraduate research provides an opportunity for students to engage in meaningful and impactful academic work, achieving optimal outcomes can be challenging. Effective supervision requires substantial time, effort, and professional commitment, particularly given the diversity among students in terms of academic background, abilities, expectations, and research approaches. Consequently, supervisors must adopt flexible strategies to accommodate individual differences while maintaining high academic standards.

Furthermore, universities adopt various supervision models, including single, joint, and panel supervision, often guided by institutional policies. However, in some cases, the absence of clearly defined policies results in subjective allocation practices managed by departmental coordinators. While co-supervision offers advantages such as exposure to diverse expertise and perspectives, inconsistencies in allocation processes may affect the overall quality and effectiveness of supervision (Elishiba & Kimani, 2014).

1.2 Background of the Study

Research supervision is a vital process in completing undergraduate studies. In this process, the supervisor is designated to facilitate the student's research development by leveraging the institution's good resources. At this stage, students faced a problem of being allocated to a supervisor whose research interest area did not match, and one of the most common complaints from research students concerns infrequent or erratic contact with supervisors, who may be too busy with administrative or teaching responsibilities, have too many students, or be away from the university too often.

There are several definitions associated with supervision, among which are foreperson, overseer, coach, facilitator, and coordinator. In the context of the undergraduate studies programme, the most suitable definition of a supervisor is one in a position of trust, to guide the student in the course of the research work, while being held responsible for the quality of work and performance in line with the research guidelines and expectations of the Graduate School/Board and the university (Mutula, 2009).

Undergraduate allocation is the systematic distribution of undergraduate students to supervisors for effective monitoring and supervision based on various criteria. Elishiba and Kimani (2014) argued that the appointment of supervisors for each undergraduate student may, in itself, not be a challenge where departments and faculties have policies guiding the process. However, in the absence of clear policies and guidelines, there are situations where supervisors are allocated to students without consultation with both parties. In other cases, undergraduate students approach the staff members they think would be good supervisors, often on advice from the other students or lecturers in the departments. The real challenge to the quality of the supervision comes with the compatibility of the students and the supervisor for ease during the research process.

1.3 Statement of Problem

The allocation of undergraduate students to project topics and supervisors remains a critical yet challenging process in many higher education institutions, where it is often conducted manually and lacks standardization. Existing approaches, including traditional Student-to-Project Allocation (SPA) models, frequently adopt one-sided preference structures and fail to adequately incorporate the mutual preferences and constraints of both students and supervisors. As a result, these methods often produce unstable or suboptimal matching, leading to inefficiencies and reduced effectiveness in the supervision process.

In response to these limitations, this study seeks to develop and implement a Genetic Programming-based model to optimize the allocation of undergraduate project topics. The proposed approach aims to enhance allocation efficiency, ensure fairness, and improve alignment between students' research interests and supervisors' expertise.

1.4 Motivation for the study

The manual approach to project allocation in higher education institutions requires critical evaluation, particularly given the significance of project work as a mandatory component of undergraduate degree programmes. Evidence from the existing case study indicates that allocation is often conducted using arbitrary criteria, such as sequential assignment based on students' matriculation numbers or the seniority of supervisors.

However, such practices are associated with several limitations, including subjective bias, inefficient allocation, duplication of project topics, and incomplete project execution. Consequently, these challenges may adversely affect the quality of supervision, student motivation, and overall research outcomes. In response to these limitations, this study proposes a

model aimed at improving the allocation of project topics by aligning students' research interests with appropriate domains and supervisory expertise.

1.5 Aim and Objectives

1.5.1 Aim

According to Abraham et al. (2007) Student Project Allocation (SPA) is an example of a two-sided matching problem, a large and very general class of problems whose input includes a set of participants that can be partitioned into two disjoint sets A and B (in this case A is the set of students and B is the set of projects), and we seek to match members of A to members of B , i.e. to find a subset of $A \times B$, subject to various criteria.

This study aims to design a system that allocates a given non-empty set of project topics to undergraduate students and matches supervisors based on research interest using a genetic programming (GP) approach for more effective and efficient topic allocation and supervision.

1.5.2 Objectives

- i. To design a system that dynamically allocates project topics to undergraduate students from a given non-empty set of project topics using genetic programming.
- ii. To develop a system that indirectly allocates a project supervisor to an undergraduate student based on the student's research interest (allocated topic) and the supervisor's research domain
- iii. To develop a system with increased efficiency and effectiveness in undergraduate project topics allocation to improve the rate and speed at which project topics are systematically allocated to undergraduate students.

1.6 Methodology of the study

This study adopts the Soft Computing technique as opposed to traditional computing, which deals with approximate models and gives solutions to complex real-life problems. Unlike hard computing, soft computing is tolerant of imprecision, uncertainty, partial truth, and approximations.

The following procedures will be taken:

- i. **System Requirement:** Research project topics and their domain shall be gathered from various supervisors (who are lecturers from the student's department).
- ii. **Specification:** The system function (operation) shall include, but not be limited to, saving of project topics, saving of student details and their research interest, saving of research domains, and allocating projects to students.
- iii. **System Model:** The system shall be modelled in relation to the specification provided above using a well-defined Unified Modelling Language (UML).
- iv. **System Development:** The system component shall be implemented using a web architecture for seamless integration.

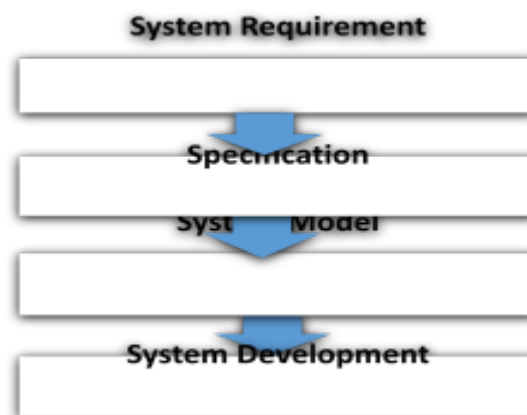


Figure 1.1: Study Methodology

1.7 Proposed Model

1.7.1 Conceptual Model

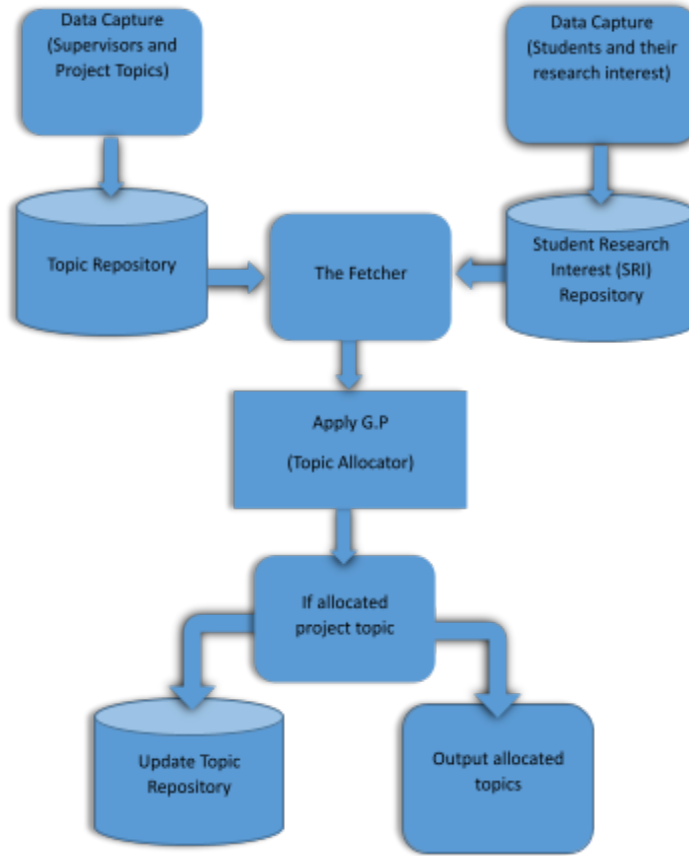


Figure 1.2: Proposed Model

1.7.2 Mathematical Model

An instance of the *Student-Project Allocation problem* (SPA) may be defined as follows:

Let $S = \{s_1, s_2, \dots, s_n\}$ be a set of *students*,

Let $D = \{d_1, d_2, \dots, d_n\}$ be a set of *domain/research interest*

Let $L = \{l_1, l_2, \dots, l_n\}$ be a set of *lecturers*, and each lecturer l_k offers a non-empty set of project topics $P = \{p_1, p_2, \dots, p_n\}$. Salami and Mamman (2016) formulated the SPA problem as follows:

$$\text{Minimize } \sum_{i=1}^l \sum_{j=1}^s C_{ij} X_{ij} \quad (1)$$

Subject to:

$$\sum_{i=1}^l X_{ij} = W_i, \quad i \in L \quad (2)$$

$$\sum_{i=1}^l X_{ij} = 1, j \in S \quad (3)$$

$$X_{ij} \in \{0, 1\}, \quad i \in L, j \in S \quad (4)$$

Where l = number of lecturers, s = number of students, W_i = number of students the i^{th} supervisor can take, C_{ij} = cost of assigning the i_{th} lecturer to the j_{th} student, and its value can be read from the entry in the i_{th} row and j_{th} column of the student preference matrix. X_{ij} is the assignment variable whose value is sought. If $X_{ij} = 1$, the i_{th} lecturer is assigned to the j_{th} student.

Since lower values of student rankings indicate better preference, the objective function in (1) is to be minimized. The constraint in (2) ensures that the number of students each lecturer supervises matches the lecturer's workload capacity. Each student should be assigned only one lecturer, as presented in (3). The constraint in (4) complements that of (3). Both constraints ensure that each student is assigned to exactly one supervisor; in other words, there is no provision for co-supervision.

1.8 Scope of the study

This research centered on a Genetic Programming (GP) approach for allocating undergraduate project topics based on students' research interest and supervisors' research domain, with consideration of the individual supervisor's capacity, while employing Soft Computing technique, taking Adekunle Ajasin University, Akungba-Akoko, Ondo State as a case study. The proposed allocation system is designed to cover the allocation of project topics to undergraduate students only. However, it did not entail postgraduate allocation, nor did it entail the actual project supervision and monitoring.

1.9 Significance of the study/expected contribution

- i. The study will introduce a systematic and data-driven approach that enhances the efficiency, fairness, and transparency of the allocation process.
- ii. The developed system will offer practical value to university administrators and academic departments by serving as a scalable and adaptable tool for managing project allocations.
- iii. The study contributes to existing knowledge in the field of optimization and decision support systems by extending the application of soft computing techniques to the Student-to-Project Allocation (SPA) problem.

1.10 Definition of Terms

GENETIC PROGRAMMING: is a technique whereby computer programs are encoded as a set of genes that are then modified (evolved) using an evolutionary algorithm.

PROJECT: This is any piece of work that is undertaken or attempted to achieve a particular result.

TOPIC (TOPICS): The subject matter or content of a conversation or discussion.

ALLOCATION: The systematic distribution of a limited quantity of resources over various time periods, products, operations, or investments.

RULE-BASED: In computer science, a **rule-based** system is a set of "if-then" statements that uses a set of assertions, to which **rules** on how to act upon those assertions are created. In software development, **rule-based** systems can be used to create software that will provide an answer to a problem in place of a human expert.

SUPERVISION: is the Monitoring and regulating of processes, delegated activities, responsibilities, or tasks.

MONITORING: This deals with observing and checking the progress or quality of (something) over a period of time

INFORMATION GATHERING: Information Gathering, otherwise called Fact-Finding, is the formal process of using research, interviews, questionnaires, and other techniques to collect/gather information about problems and requirements.

CONFLICT: Analysis is the activity of identifying the problem, understanding the problem, and understanding any constraints that may limit the solution.

GODFATHERISM: is the act of an influential member of staff assisting or favouring a particular student over the others.

CHAPTER TWO

LITERATURE REVIEW

2.1 Resource Allocation

Resource allocation is the act of distributing resources by apportioning activities and the resources required by those activities, while taking into consideration both the resource availability and the project time with respect to a particular set of principles.

In a similar context, Ajayi & Anifowose (2017) define resource allocation as a process and strategy involving an organization deciding where scarce resources should be used in the production of goods or services. A resource can be considered any factor of production, which is something used to produce goods or services. Resources include such things as labour, real estate, machinery, tools and equipment, technology, and natural resources, as well as financial resources, such as money. Resource allocation begins at strategic planning when an Organization formulates its vision and goals for the future. The vision and strategic goals are accomplished through the achievement of objectives.

2.2 Responsibilities of the supervisor

According to Augustsson and Jaldemark (2012), the supervisor(s) shall monitor, support, and direct the student's work and progress soon after the allocation of project/dissertation titles.

The responsibilities of the supervisor include:

- (i) Proposing/supervising projects/dissertations in their own subject area.
- (ii) Setting a framework for regularly scheduled progress meetings between supervisor(s) and student;
- (iii) Giving guidance about relevant literature on the topic under study and appropriate literature sources.

- (iv) Giving guidance about the formulation of an appropriate hypothesis-driven research project and focusing on the objectives of the research (if applicable).

2.3 Responsibilities of the student

Throughout the whole project/research work, the student shall/may seek advice, comments, and guidance from his/her supervisor(s) on the nature of the project work and the standard expected. Students are advised to keep a notebook for the purpose of the meeting with supervisor(s), while the supervisor(s) may wish to keep a brief record of each meeting held.

Bouki (2007) and Augustsson (2012) point out the responsibilities of the student to include:

- (i) Arranging with his/her supervisor(s) mutually agreed convenient times to discuss progress achieved (if meetings are not possible, e-mails or other forms of communication may be used).
- (ii) Bringing to the urgent attention of his/her supervisor(s) any problems (academic and personal) associated with the research progress.
- (iii) Responding to the supervisor's suggestions and/or criticisms on his/her work and progress.
- (iv) Discussing the layout of the final dissertation with the supervisor(s) before the writing-up stage.

2.4 Related Works

The Student–Project Allocation (SPA) problem has attracted significant attention in the field of optimization and decision support systems due to its complexity and practical relevance in academic environments. Fundamentally, SPA involves matching students to projects (and supervisors) based on preference structures and institutional constraints, making it a multi-objective optimization problem.

Pan et al. (2009) approached the SPA problem using a goal programming model, emphasizing its multi-objective nature. Their model sought to maximize the number of allocated projects while simultaneously optimizing student satisfaction and minimizing disparities based on academic performance indicators such as Grade Point Average (GPA). By introducing binary decision variables and preference matrices, the study provided a structured mathematical formulation of the allocation process. However, despite its robustness, the approach is limited by its deterministic nature and may struggle to adapt to dynamic or large-scale allocation scenarios where preferences and constraints frequently change.

Similarly, Dimitar (2011) explored the SPA problem using the Stable Marriage (SM) algorithm, a classical matching approach designed to ensure stability in pairings. The SM algorithm guarantees that no pair of participants would prefer each other over their assigned matches, thereby preventing instability in the allocation. Unlike goal programming approaches, this approach effectively addresses fairness and stability; it does not inherently optimize global allocation efficiency or accommodate capacity constraints in complex institutional settings.

Moussa and El-Atta (2011) extended the SPA framework by introducing a visual implementation that utilizes preference lists over student–project pairs. Their study demonstrated that incorporating structured preference lists can improve the size and stability of matchings while reducing computational complexity through efficient data structures. Nevertheless, the approach remains largely focused on representation and visualization, with limited emphasis on optimization performance or scalability.

Further advancements were made by Abraham et al. (2007), who investigated SPA models incorporating preferences from both students and lecturers. Their work introduced a two-sided matching framework where both parties provide ranked preferences, thereby

enhancing the realism and applicability of the allocation process. Although these models improve fairness and mutual satisfaction, they are still constrained by their reliance on stable matching principles, which may not always yield globally optimal solutions in terms of overall system efficiency.

In a related study, Aswhin (2014) examined the SPA problem in the context of group projects and demonstrated that the allocation problem becomes NP-complete under certain conditions. The study further showed that the problem becomes polynomially tractable when specific constraints, such as upper quotas, are limited. This highlights the inherent computational complexity of SPA problems and underscores the need for heuristic or metaheuristic approaches capable of efficiently handling large-scale instances

Kwanashie et al. (2015) introduced profile-based optimal matching techniques, leveraging Network Flow algorithms to achieve greedy and generous maximum matchings. Their approach provided an efficient means of balancing fairness and optimality while offering empirical validation of performance improvements. However, similar to other deterministic approaches, the model may be less adaptable to dynamic or highly constrained environments where multiple conflicting objectives must be addressed simultaneously.

Fabiyi et al. (2016) developed a web-based student project allocation system using matching algorithms, emphasizing the importance of stability in assignments. While the system demonstrated practical applicability, its reliance on traditional matching techniques limits its ability to explore alternative optimal solutions or adapt to complex preference structures.

In contrast, Salami and Mamman (2016) proposed a Genetic Algorithm (GA)-based approach to the SPA problem, incorporating student preferences and supervisor workload constraints. Their findings indicated that GA outperformed conventional optimization methods

by generating multiple feasible solutions, thereby allowing decision-makers to select the most suitable allocation. Moreover, the study highlighted the effectiveness of problem-specific genetic operators in improving solution quality. Nevertheless, the approach did not fully explore the broader potential of evolutionary computation techniques, particularly in handling multi-objective optimization and dynamic allocation scenarios.

2.4.1 Table 1.1: Cited related works

S/N	AUTHOR	ARTICLE	AIM	APPROACH	ACHIEVEMENT	ATTENTION
1	Manlove and O'Malley (2008)	Student-Project Allocation with preferences over Projects	To assign students to projects subject to preference lists and capacity constraints.	Approximation Algorithm	The approach presented a better solution when compared to linear programming	The Model only provides an approximate solution with a performance guarantee of 2.
2	Pan et al. (2009)	Multi-criteria Student Project Allocation: A Case Study of Goal Programming Formulation with DSS Implementation	Solve a multi-criteria SPA problem imposed by the Department of Mathematics of the University of Hong Kong.	Goal Programming	When the model was compared with manual solutions given by the department, the GP model has produced an effective and efficient distribution plan for all projects being allocated to students with the	It is limited to the University of Hong Kong

					best priority costs and the highest possible GPA's.	
3	El-Sherbi ny <i>et al.</i> (2012)	An Artificial Immune Algorithm with Alternative Mutation Methods: Applied to the Student Project Assignment Problem	To solve the student project allocation problem subject to students' preferences over projects.	Artificial immune algorithm	The results of the study showed that the artificial immune algorithm found better quality solutions compared to the Genetic algorithm	It requires more time when compared to the genetic algorithm
4	Fabiyi <i>et al.</i> (2016)	Development of the Student Project Allocation System Using Matching Algorithm	To solve the problem of allocating project students to appropriate project supervisors.	Matching Algorithm	The algorithm provides a solution to the allocation of students with an area of interest to A supervisor who specializes in the area of research of students.	It does not allow student to provide their desired project topic.
5	Salami and Mamman (2016)	A Genetic Algorithm for Allocating Projects Supervisors to Students	A genetic algorithm (GA) for assigning project supervisors to students, taking into account the students' references for lecturers as	Genetic Algorithm	The GA successfully assigned the Most preferred project supervisors are subject to the workload constraints of the supervisors.	

			well as lecturers' capacities.			
6	Sam and Ian (2016)	Solving the Student Project Allocation Problem	To solve the student project allocation problem where student specify their project of interest and supervisors select the project they wish to supervise.	Matching Algorithm (specifically Bellman-Ford single-source shortest paths algorithm)	A matching algorithm was developed and implemented an output that optimally matches students and projects, such that no student could obtain a better project in the same maximal matching.	It requires Load balancing for lecturers, and the computational cost of the algorithm is high.
7	Lei and Chim (2017)	A Web-based Project Allocation System	To convert SpaGroup to solve the spa with group projects.	An algorithm called the SpaGroup is designed, which aims to generate a stable matching	The algorithm successfully generates the <i>optimal matching</i> between students and projects based on the preferences and the capacities.	The performance of the Spa-Group algorithm is greatly reduced When the input is vast, say millions of students or projects

2.5 Appraisal of Literature

Collectively, these studies reveal that while significant progress has been made in addressing the SPA problem, existing approaches tend to focus either on stability, such as matching algorithms or optimization like the Network Flow algorithm, with limited integration

of both objectives. Furthermore, many traditional models lack scalability, adaptability, and the ability to handle complex, real-world constraints effectively.

Therefore, there is a clear need for more robust and flexible approaches that can simultaneously optimize allocation efficiency, ensure fairness, and accommodate dynamic preference structures. In this context, evolutionary computation techniques, particularly Genetic Programming, offer a promising alternative due to their ability to explore large solution spaces and generate near-optimal solutions under multiple constraints. This study builds upon existing work by proposing a Genetic Programming-based model to enhance the efficiency, scalability, and fairness of undergraduate project allocation systems.

CHAPTER THREE

METHODOLOGY AND MODEL

3.1 Methodology

This study adopts the use of Soft Computing Technique. Soft computing deals with imprecision, uncertainty, partial truth, and approximation to achieve practicability, robustness, and low solution cost (Gupta and Kulkarni, 2013). Soft computing is not a single method, but instead it is a combination of several methods, such as fuzzy logic, neural networks, genetic algorithms, and genetic programming. These methods are not competitive, but are complementary to each other and can be used together to solve a given problem. It can be said that soft computing aims to solve complex problems by exploiting the imprecision and uncertainty in decision-making processes (Gupta and Kulkarni, 2013). Soft Computing is a new multidisciplinary field, to construct a new generation of Artificial Intelligence, known as Computational Intelligence.

Soft computing was chosen for this study because of its features such as intelligent control, nonlinear programming, optimization, and decision-making support, which are key requirements in resource allocation.

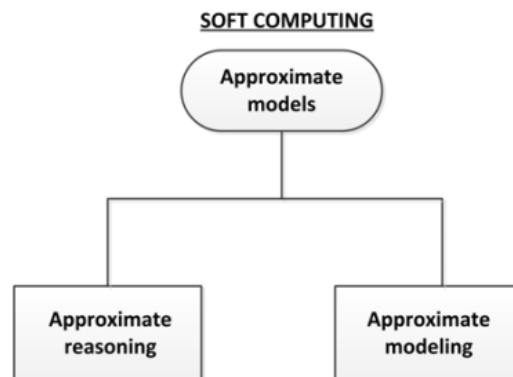


Figure 3.1: Soft Computing Problem-Solving Approach

Image Source: (Ibrahim 2016) 12th International Conference on Application of Fuzzy Systems and Soft Computing, ICAFS 2016, 29-30 August 2016, Vienna, Austria

3.2 Methodological Details

The following high-level procedures were carried out in executing this project:

- v. **System Requirement:** Research project topics and their domain shall be gathered from various supervisors (who are lecturers from the student's department).
- vi. **Specification:** The system function (operation) shall include, but not be limited to, saving of project topics, saving of student details and their research interest, saving of research domains, and allocating projects to students.
- vii. **System Model:** The system shall be modelled in relation to the specification provided above using a well-defined Unified Modelling Language (UML).
- viii. **System Development:** The system component shall be implemented using a web architecture for seamless integration.

3.2.1 System Requirement

Extensive and adequate information was gathered in the field of this study, some of these include extensive literature review of past research works in this domain of study. Also, experts were consulted in this field of study, and information was extracted on how the process is carried out and the needed measures to take into consideration in the design of the system. Moreover, sample project topics and domains were collected from various supervisors who are lecturers at the Department of Computer Science in Adekunle Ajasin University, Akungba, Akoko.

3.2.2 Specification

The system function shall include, but is not limited to, the storage of project topics, student details, research domains, and research interests. Additionally, the allocation of the project topic to students. Furthermore, this phase is required to provide a clear and formal

definition of system requirements, thereby facilitating efficient system design and offering a structured basis for subsequent testing and verification.

3.2.3 **System Model**

Following the specification phase, a detailed system modelling process was undertaken using Microsoft Visio, a Computer-Aided Design (CAD) tool, to represent the structural and functional components of the system. Furthermore, this phase is essential for facilitating systematic analysis, validation, and early detection of potential design inconsistencies before implementation. In this study, the Unified Modeling Language (UML) was employed, incorporating context diagrams, activity diagrams, use-case diagrams, and sequence diagrams to provide a comprehensive representation of the system architecture and interactions.

3.2.4 **System Development**

Following a critical evaluation of the data and system modelling phase, a suitable system was developed using PHP and MySQL for backend development, and JavaScript, HTML, and CSS for frontend implementation. The system was structured into distinct functional modules, including supervisor, student, and administrator components, to enhance modularity, facilitate efficient development, and support systematic testing.

3.3 Architectural Model

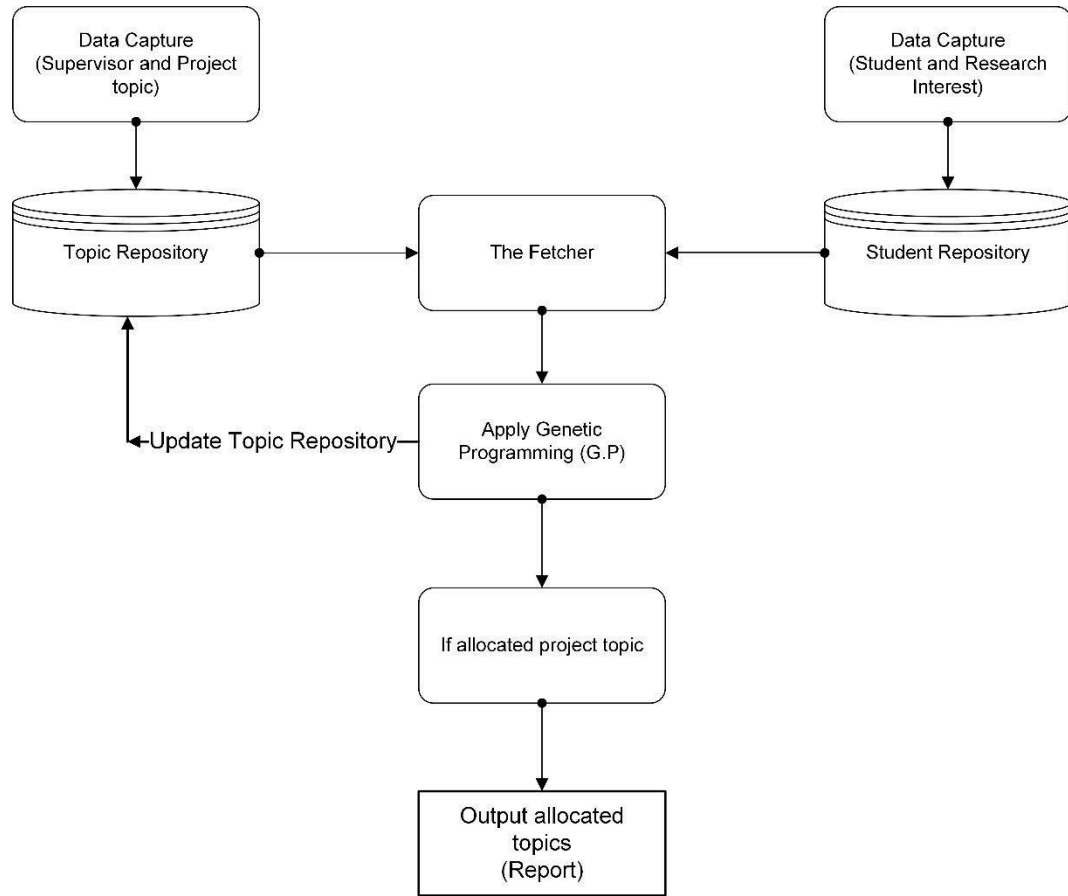


Figure 3.2: Proposed Architectural Model

3.3.1 Detailed Analysis of the Model

The model employed in this study consists of the following components (elements):

- i. Data Capture
- ii. Topic Repository
- iii. Student Repository
- iv. The Fetcher
- v. Apply Genetic Programming (GP)
- vi. Updated Topic Repository

3.3.1.1 Data Capture

This component handles the retrieval/capturing of data from the users (students, supervisors, and admins), such as project topics, research domains, research interests, etc. This covers all stages from computer inputting, data verification, data preparation, sorting, control, and transmission. The following algorithm will be used/implemented.

Data Capture Algorithm:

- i. Get user-submitted/inputted data
- ii. Verify inputted/submitted data
- iii. Validate input parameters
- iv. Filter/sanitize input parameter
- v. Transmit the filtered data to the respective repository (Topic Repository or Student Repository)

3.3.1.2 Topic Repository

The topic repository is a database of tables for a collection of supervisors' topics and research domains. Additionally, type checking, column binding, and storage of research topics and domains into the topic repository are done using the following algorithm.

Research topic storage algorithm:

- i. Get the filtered captured data
- ii. Check/validate the retrieved parameter's data type
- iii. Map/bind each parameter's value to the appropriate column
- iv. Save the mapped parameters to the topic repository

3.3.1.3 Student Repository

The student repository is a collection of students' data and research interests that are fundamental to the system.

Research topic storage algorithm:

- i. Get the filtered captured data
- ii. Check/validate the retrieved parameter's data type
- iii. Map/bind each parameter's value to the appropriate column
- iv. Save the mapped parameters to the student repository

3.3.1.4 The Fetcher

The student fetcher retrieves student data such as student ID, phone number, email, and research interest from the student repository, and feeds it to the genetic programming component, so also is the topic fetcher fetches the research topic and the domain, and then feeds it to the genetic programming component.

3.3.1.5 Apply Genetic Programming (G.P)

This is the main component of the system, which uses genetic programming to allocate (assign) a project topic to a student based on the research interest specified by the student during data capture. The following genetic programming is defined by the Genetic Programming Algorithm as defined by (Spector *et al.*, 2002):

Step 1: Create an initial population of random individuals.

Step 2: Test the “fitness” of each program.

Step 3: If an individual program yields an optimal match, then stop and report success.

Step 4: Otherwise, create a new population from the more fit individuals, using “genetic operators” such as reproduction, mutation, and crossover.

Step 5: Replace the old population with the new population and return to Step 2.

3.3.1.6 Update Topic Repository

This component updates the topic repository by removing the allocated project topic from the repository into the allocated topic repository, and then mapping the student to their respective supervisor. The following algorithm will be implemented during the topic repository stage.

Step 1: *Get allocated topic ID*

Step 2: Fetch the allocated topic from the topic repository using the ID

Step 3: Insert the retrieved allocated topic into the allocated topic repository

Step 4: Remove the allocated topic from the topic repository

Step 5: Insert the student ID into the supervisor repository

CHAPTER FOUR

SYSTEMS DESIGN

4.1 System Design

This chapter draws the blueprints for constructing and implementing the new system. Here, the various design specifications of the new system are examined. However, this study employs the MVC (Model, View, and Controller) design pattern.

4.2 Design Specification:

In systems engineering, a specification is a document that clearly and accurately describes the essential technical requirements for items, materials, or services, including the procedures by which it can be determined that the requirements have been met.

According to Sommerville (2011), System requirements are expanded versions of the user requirements that are used by software engineers as the starting point for the system design. They add detail and explain how the user requirements should be provided by the system. They may be used as part of the contract for the implementation of the system and should therefore be a complete and detailed specification of the whole system.

The following specifications will be highlighted and discussed in sections:

- i. Database Specification
- ii. Interface Specification
- iii. Technical Specification
- iv. Algorithmic Specification

4.2.1 Database Specification

MySQL 5.1.19 was used as the database for this study. It is a high-end relational database management system. Below are the database schemas for the various database files used.

Table 4.1: Student Record Schema

This table contains student records, including their research interests.

Field	Field Maximum Length	Data Type	Key Attribute/Index
matric_number	9	INT	PK (Primary Key)
first_name	10	VARCHAR	NON (Non Key Attribute)
Last_name	20	VARCHAR	NON (Non Key Attribute)
Email	40	VARCHAR	NON (Non Key Attribute)
Session	10	VARCHAR	NON (Non Key Attribute)
research_interest	50	VARCHAR	NON (Non Key Attribute)
created_date	Null	TIMESTAMP	NON (Non Key Attribute)

Table 4.2: Supervisor Record Schema

This table contains the supervisor's records, including their research domain.

Field	Field Maximum Length	Data Type	Key Attribute/Index
supervisor_id	50	INT	PK (Primary Key)
Title	4	VARCHAR	NON (Non Key Attribute)
first_name	20	VARCHAR	NON (Non Key Attribute)
last_name	25	VARCHAR	NON (Non Key Attribute)
phone_number	11	VARCHAR	UNIQUE
Email	50	VARCHAR	UNIQUE
Password	50	VARCHAR	NON (Non Key Attribute)

Table 4.3: Project Topic Schema

This table contains records for unallocated project topics.

Field	Field Maximum Length	Data Type	Key Attribute/Index
Topic_id	255	INT	PK (Primary Key)

supervisor_id	50	INT	FK (Foreign Key)
Topic	255	VARCHAR	NON (Non Key Attribute)
Domain	50	VARCHAR	NON (Non Key Attribute)
added_date	Null	TIMESTAMP	NON (Non Key Attribute)

Table 4.4: Allocated Project Topics Schema

This table contains records for unallocated project topics.

Field	Field Maximum Length	Data Type	Key Attribute/Index
Allocate_id	255	INT	PK (Primary Key)
topic_id	255	INT	FK (Foreign Key)
Matric_no	10	INT	FK (Foreign Key)
allocated_date	255	VARCHAR	NON (Non Key Attribute)

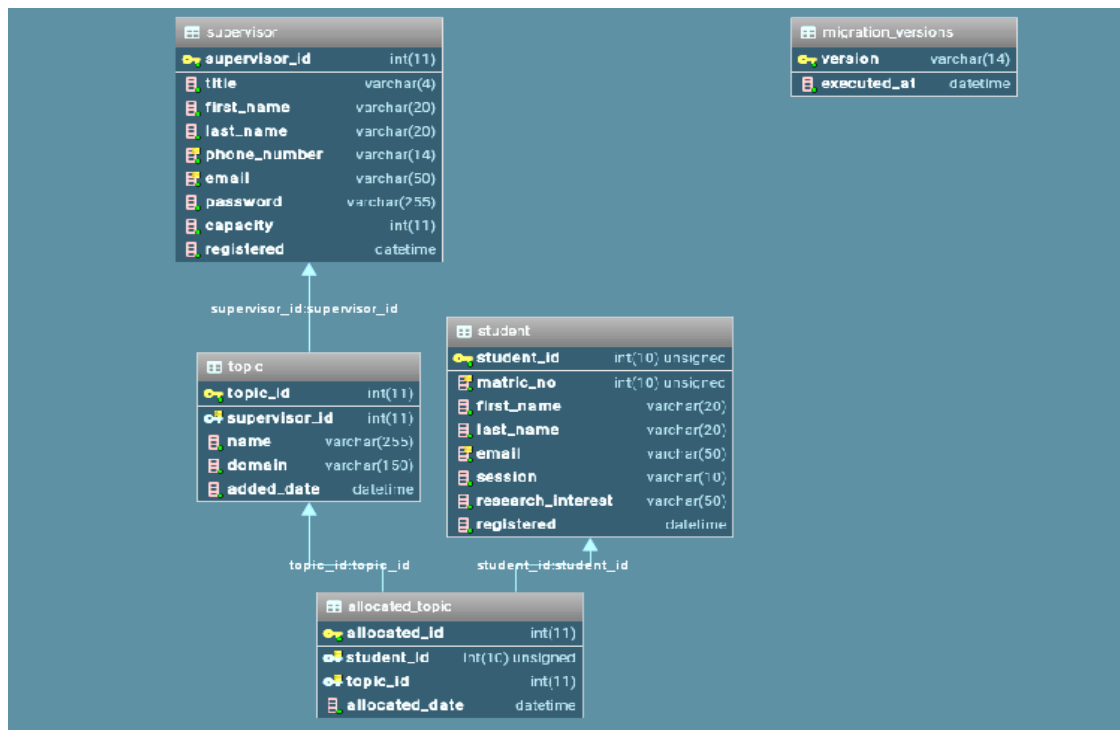


Figure 4.1: Entity Relationship Diagram (ERD)

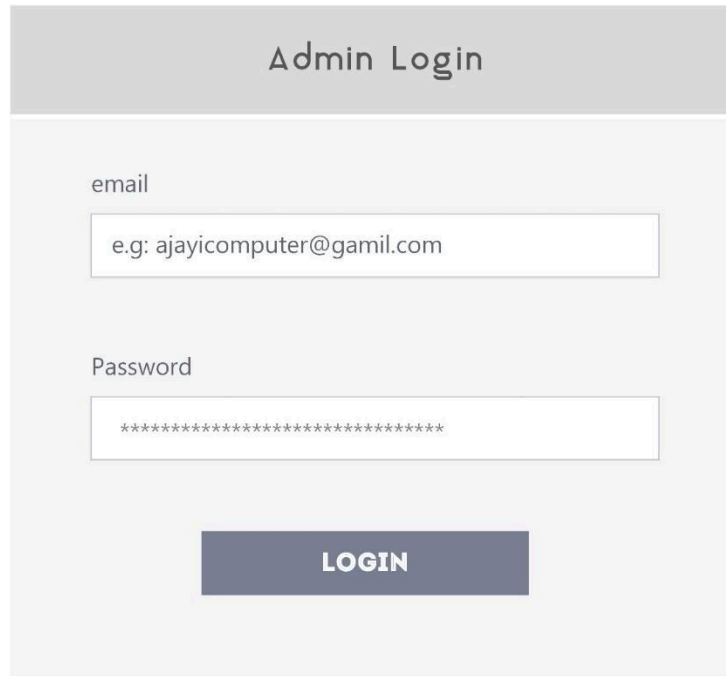
4.2.2 Interface Specification

Usability is one of the key issues in designing a system. No matter how sophisticated a system might be, if it is not usable by the intended users, then the system becomes useless. Hence, much effort is invested in the aspect of human-computer interface design when developing a new system. In this study, various interfaces have been designed to allow easy interaction and operation of the application in order to actualize the purpose for which it was developed. The design of some of the interfaces is given below, while the screenshot of the program interface is given in the next chapter of this study.

According to Sommerville (2011), Interface design is concerned with specifying the details of the interface to an object or to a group of objects. This implies defining the signatures and semantics of the services that are provided by the object or by a group of objects. Interfaces can be specified in the UML using the same notation as a class diagram.

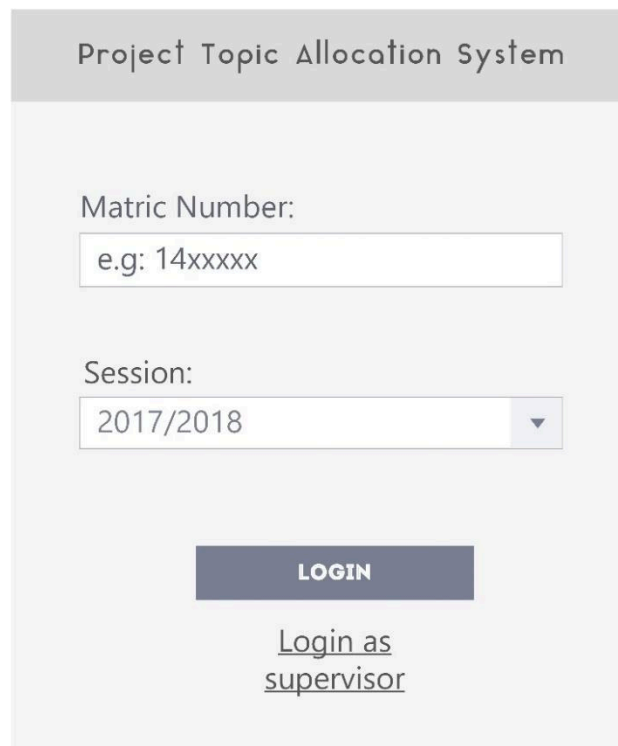
According to Bern and Allen (2010), the focus of interface specification is for developers to communicate clearly and precisely about increasingly lower-level details of the system. The interface specification activities of object design include:

- i. identifying missing attributes and operations
- ii. specifying type signatures and visibility
- iii. specifying invariants
- iv. specifying preconditions and postconditions.



The Admin Login interface features a grey header with the text "Admin Login". Below the header, there are two input fields. The first is labeled "email" and contains the placeholder text "e.g: ajayicomputer@gamil.com". The second is labeled "Password" and contains a series of asterisks. Below these fields is a dark blue button with the text "LOGIN" in white capital letters.

Figure 4.2: Admin Login Interface



The Student's Login interface features a grey header with the text "Project Topic Allocation System". Below the header, there are two input fields. The first is labeled "Matric Number:" and contains the placeholder text "e.g: 14xxxxx". The second is labeled "Session:" and contains the text "2017/2018" with a dropdown arrow. Below these fields is a dark blue button with the text "LOGIN" in white capital letters. Below the button is a link that says "Login as supervisor".

Figure 4.3: Student's Login Interface

Home Page/Landing Page: This is the first page that will be returned or served to the client when the web page is requested.

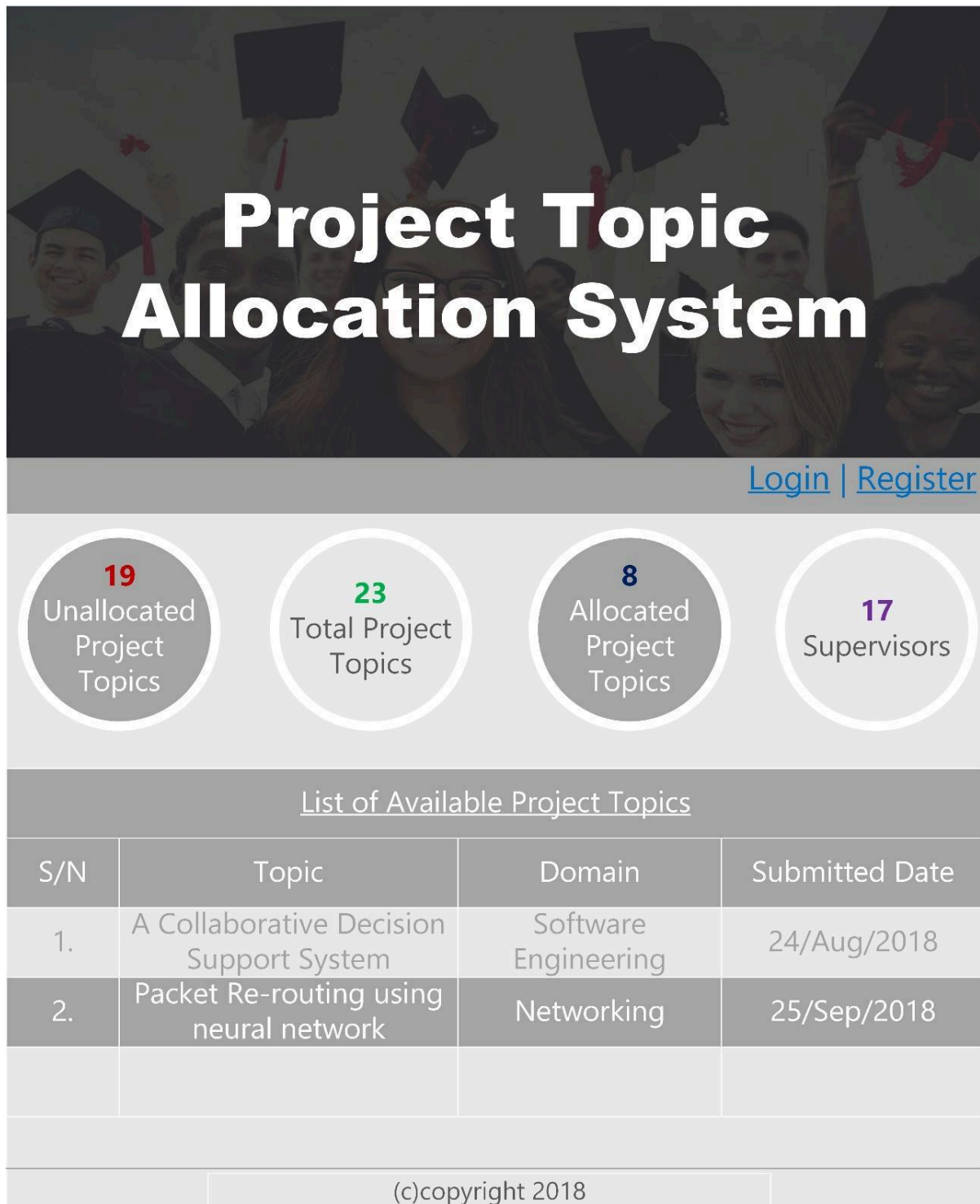


Figure 4.4: Home Page/Landing Page Interface

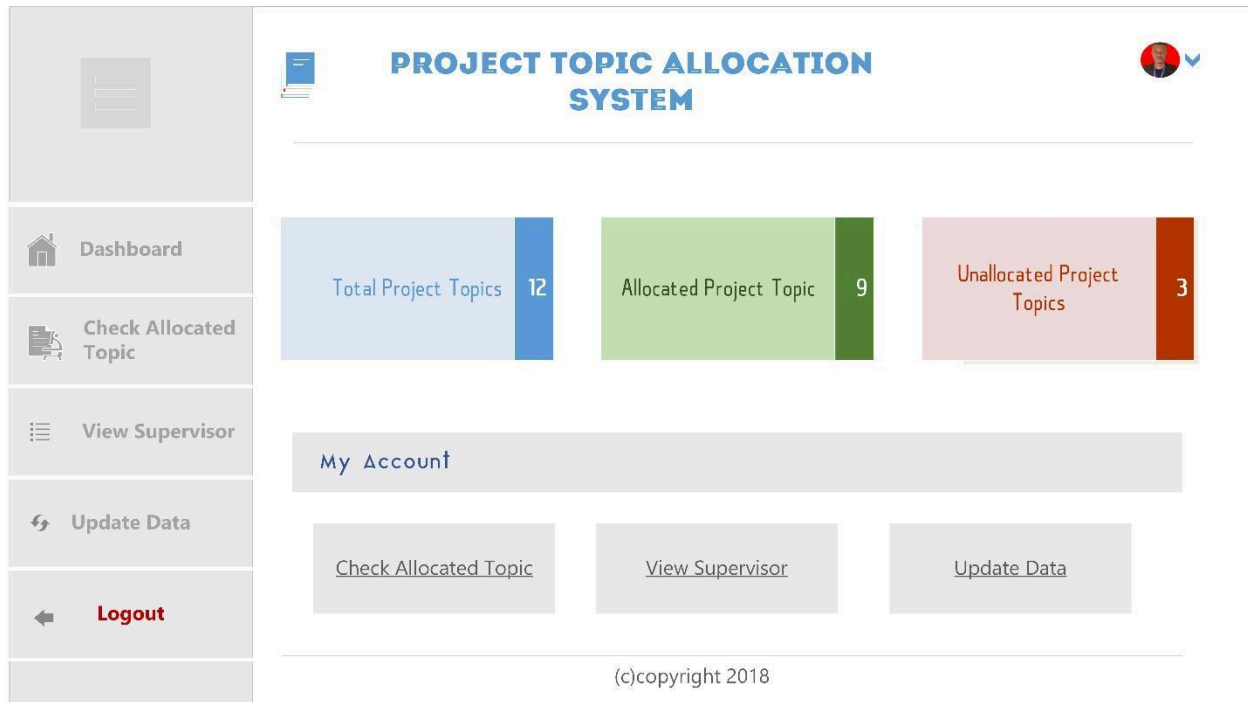


Figure 4.5: User's Dashboard Interface

Register as Student

Matric Number:

First Name:

Session:

Last Name:

Research Interest:

E-mail:

REGISTER

[Register as Supervisor](#)

Figure 4.6: Register Page Interface



- Dashboard
- Check Allocated Topic
- View Supervisor
- Update Data
- Logout

PROJECT TOPIC ALLOCATION SYSTEM

Dashboard > Check Allocated Topic (140404050)

Topic Detail:

Topic : Agentic Programming Approach for Allocating Undergraduate Project Topics

Domain : Software Engineering, Artificial Intelligence

Submitted Date : 23/Mar/2018.

Supervisor's Info:

Title : Mr.

Name : Ajayi O.

E-mail : ajayicomputer@gmail.com.

Phone : 08137044500.

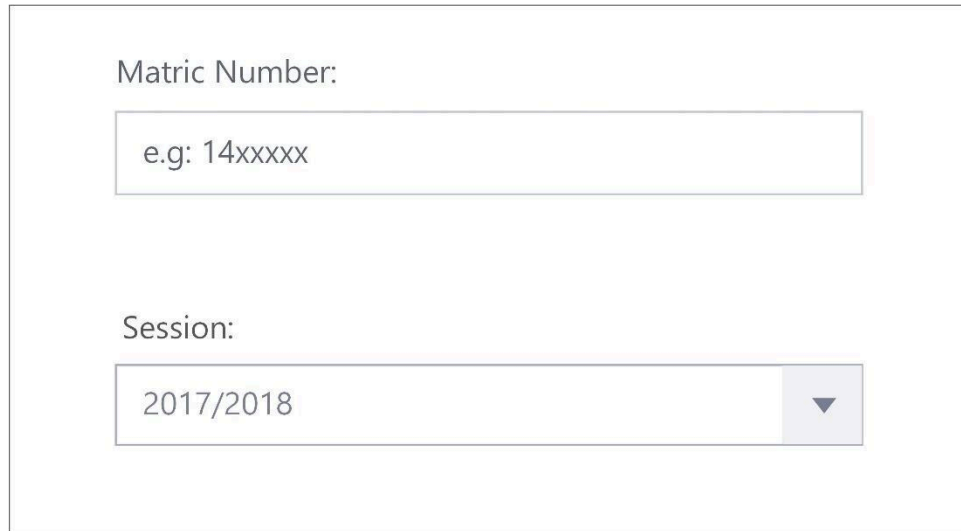
(c)copyright 2018

Figure 4.7: Check Allocated Topic Page Interface

The figure below depicts students' input parameters

Matric Number: e.g: 14xxxxx	First Name: e.g: John
Session: 2017/2018	Last Name: e.g: Micheal
Research Interest	E-mail e.g: michealJohn@aaua.edu.ng

Figure 4.8: Student's Input Parameters (Registration)



Matric Number:

e.g: 14xxxxx

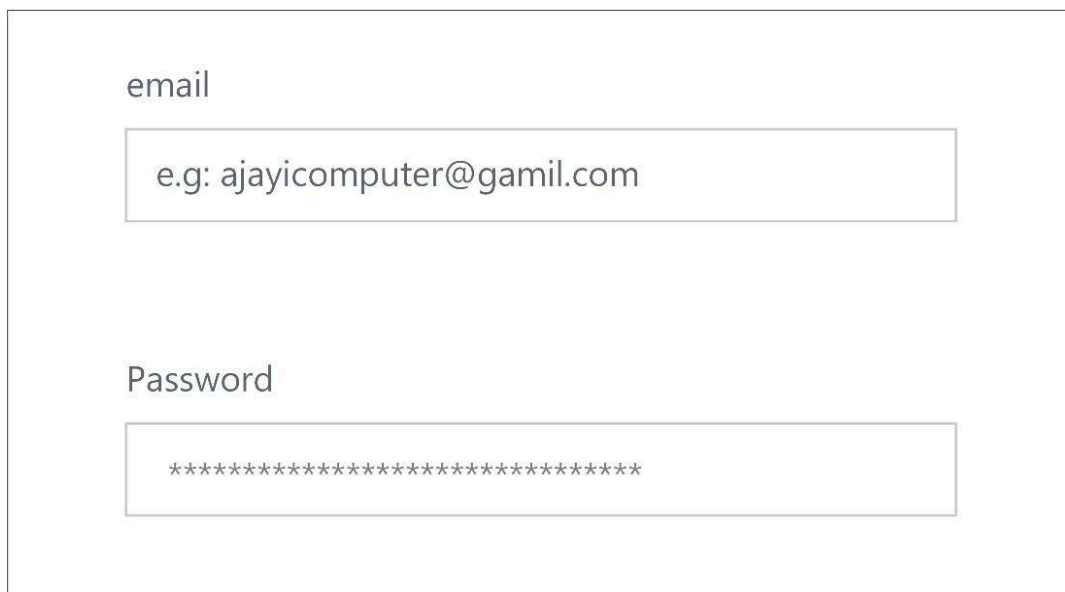
Session:

2017/2018 ▼

This figure shows a login form for a student. It contains two input fields. The first is labeled 'Matric Number:' and has a placeholder text 'e.g: 14xxxxx'. The second is labeled 'Session:' and has a dropdown menu with '2017/2018' selected and a downward arrow icon.

Figure 4.9: Student's Input Parameters (Login)

The figure below depicts the supervisor's input parameters



email

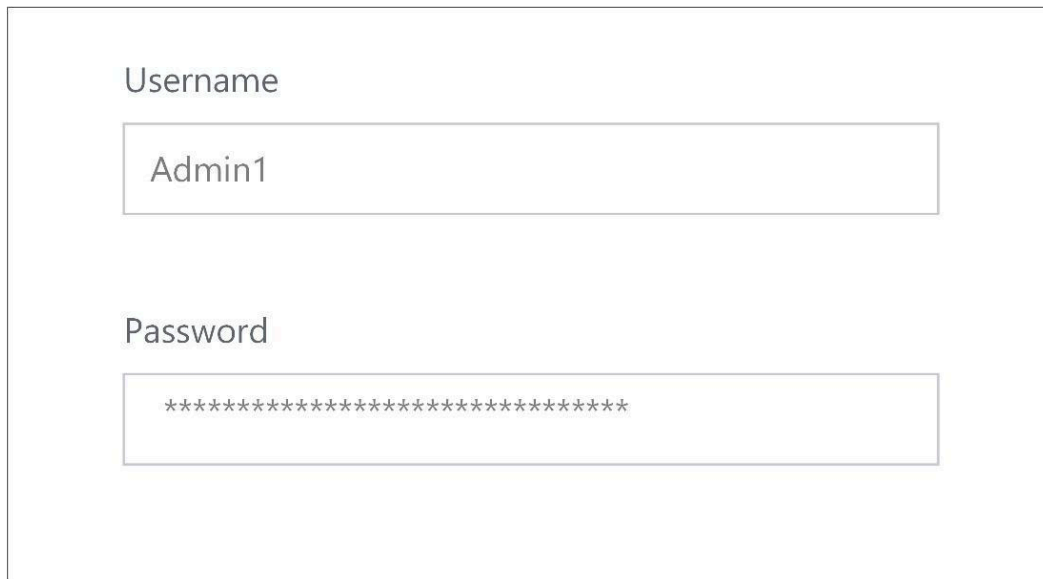
e.g: ajayicomputer@gamil.com

Password

This figure shows a login form for a supervisor. It contains two input fields. The first is labeled 'email' and has a placeholder text 'e.g: ajayicomputer@gamil.com'. The second is labeled 'Password' and has a placeholder text consisting of ten asterisks '*****'.

Figure 4.10: Supervisor's Input Parameters

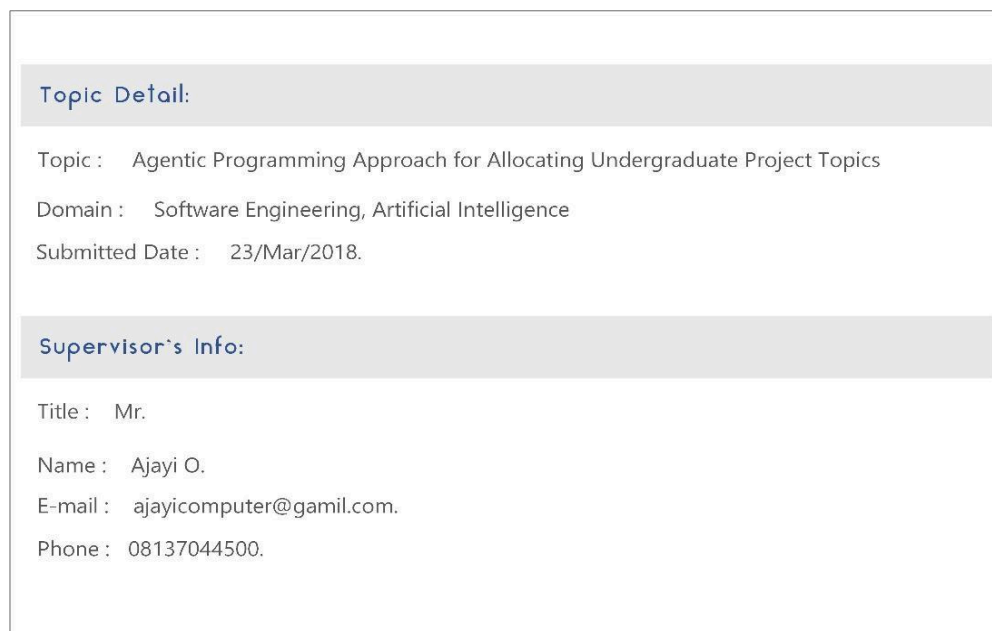
The figure below depicts Admin's input parameters



A screenshot of an admin login interface. It features two input fields. The first field is labeled 'Username' and contains the text 'Admin1'. The second field is labeled 'Password' and contains a series of asterisks '*****' to represent a masked password. The fields are simple rectangular boxes with thin borders.

Figure 4.11: Admin's Input Parameters

The figure below displays a sample output of the result after a successful allocation of undergraduate project topics.



A screenshot of a web application output interface. It is divided into two main sections. The first section, titled 'Topic Detail:' in blue text on a light gray background, contains the following information: 'Topic : Agentic Programming Approach for Allocating Undergraduate Project Topics', 'Domain : Software Engineering, Artificial Intelligence', and 'Submitted Date : 23/Mar/2018.'. The second section, titled 'Supervisor's Info:' in blue text on a light gray background, contains the following information: 'Title : Mr.', 'Name : Ajayi O.', 'E-mail : ajayicomputer@gamil.com.', and 'Phone : 08137044500.'. The text is in a standard sans-serif font.

Figure 4.12: Output interface

Below are UML class diagrams for the various packages:

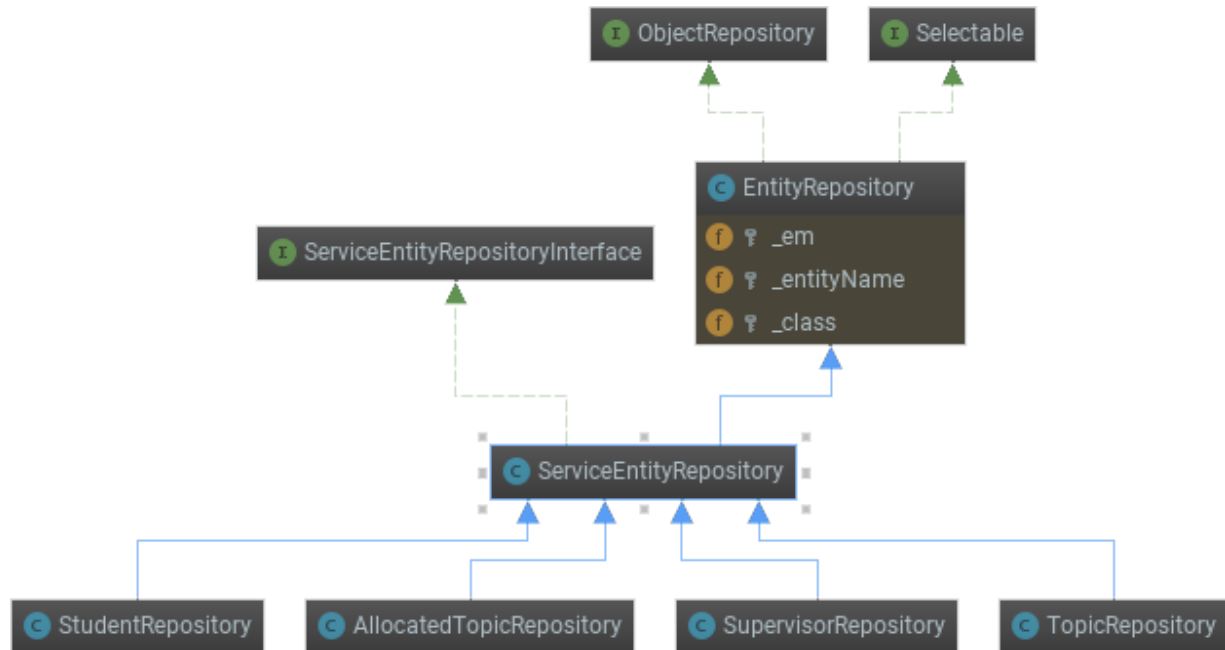


Figure 4.13: UML class diagram for repository package

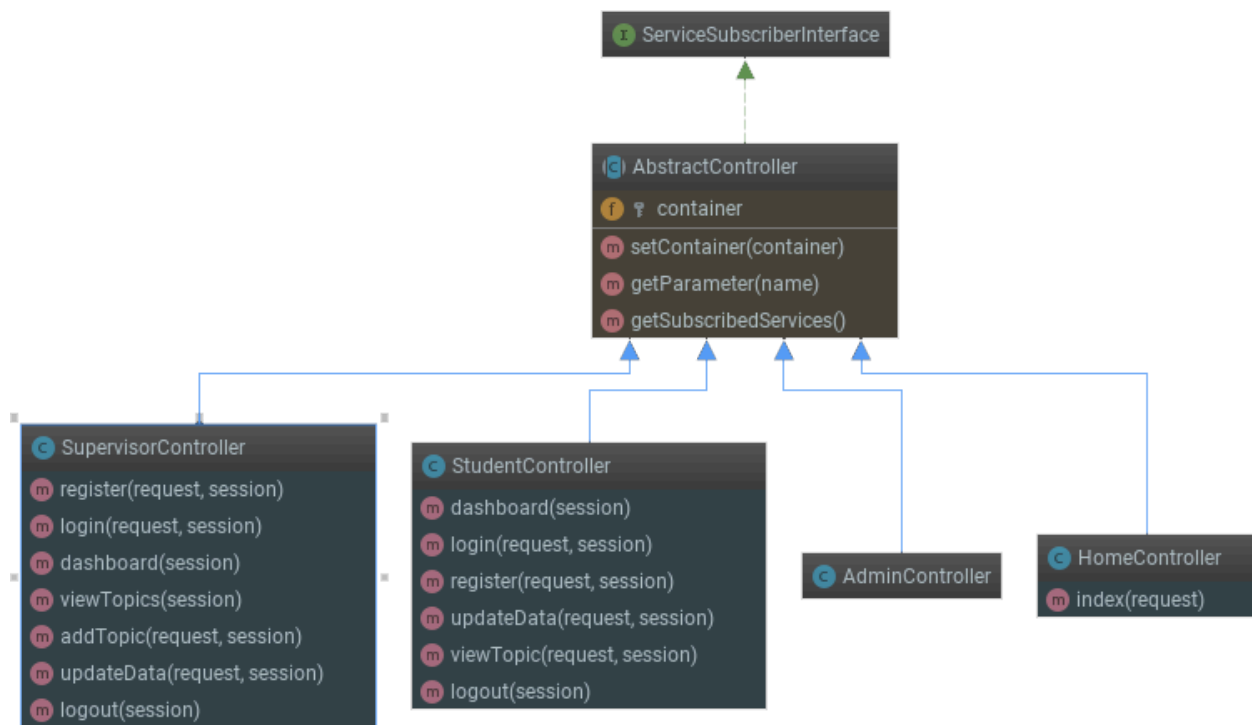


Figure 4.14: UML class diagram for controller package

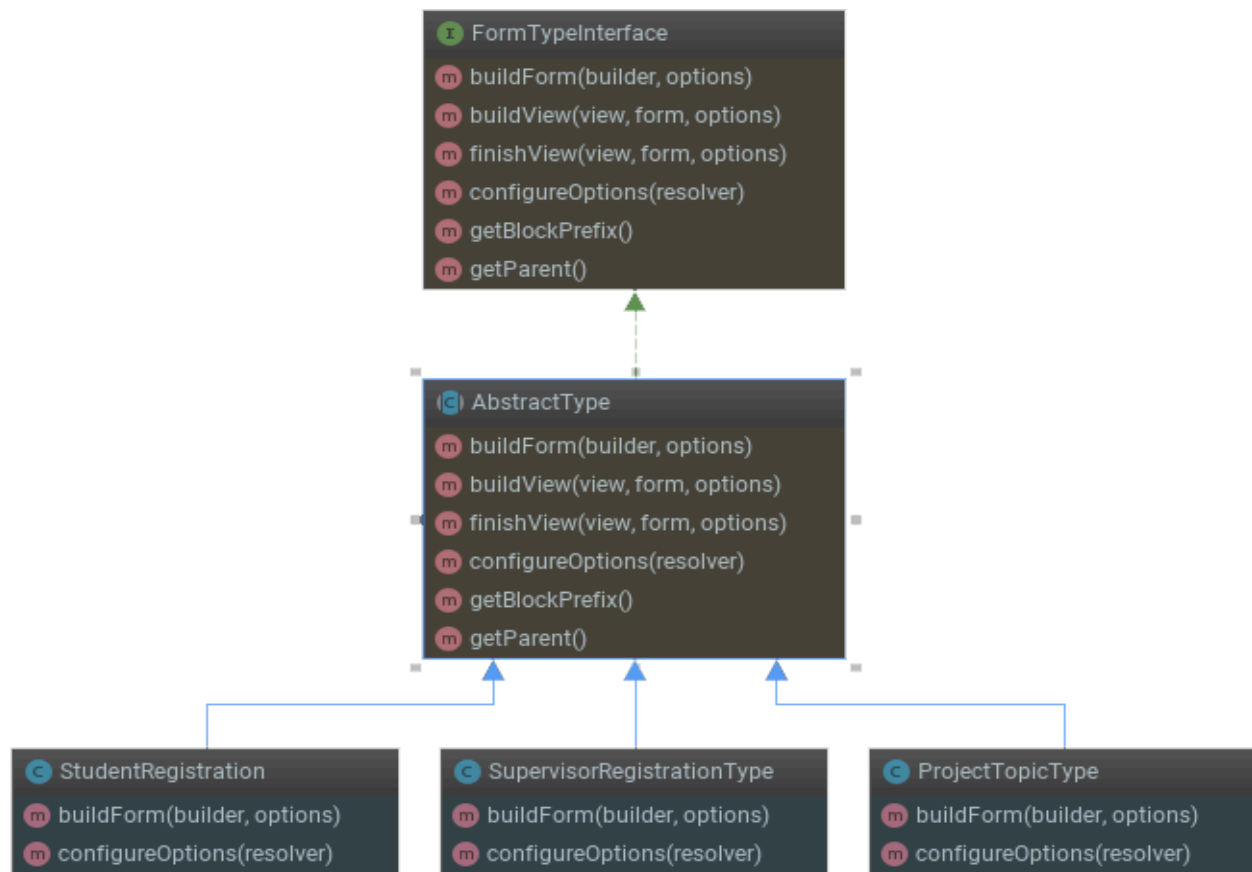


Figure 4.15: UML class diagram for form package

4.2.3 Technical Specification

A technical specification is a specification that defines a set of requirements that a product or software must meet or exceed. A product or software that does not *meet all* of the specifically expressed requirements does not meet the specification, and it is often referred to as being out of specification or "out of spec".

The system requirements shall include, but not be limited to;

- i. Saving of project topics, student details, research interest, research domains, and allocated topic using MySQL 5.1.19.
- ii. Security and encryption of students' and supervisors' data using the Symfony security pack.

- iii. Separation of concerns using MVC (Model, View, Controller) design pattern by employing a PHP framework called “Symfony”.

4.2.4 Algorithmic Specification

An algorithm is a set of instructions or procedures designed for solving a problem, based on conducting a sequence of specified actions. A computer program can be viewed as an elaborate algorithm. In mathematics and computer science, an algorithm usually means a small procedure that solves a recurrent problem.

Below are the major algorithms to be implemented or employed in the implementation of the system:

Data Capture Algorithm:

Step 1: Get the user-submitted/inputted data

Step 2: Verify inputted/submitted data

Step 3: Validate input parameters

Step 4: Filter/sanitize input parameter

Step 5: Transmit the filtered data to the respective repository (Topic Repository or Student Repository).

Research topic storage algorithm:

Step 1: Get the filtered captured data

Step 2: Check/validate the retrieved parameter's data type

Step 3: Map/bind each parameter's value to the appropriate column

Step 4: Save the mapped parameters to the topic repository.

Genetic Programming Algorithm

Genetic Programming (GP) is the core algorithm employed in this study for the dynamic allocation of project topics to undergraduate students. It is a subset of evolutionary computation and a variant of evolutionary algorithms, closely related to techniques such as Genetic Algorithms, Evolutionary Strategies, and Evolutionary Programming (Jason, 2011). GP is specifically designed to evolve computer programs or solution representations automatically through iterative optimization processes.

According to Walker (2001), Genetic Programming is a method that generates and evolves computer programs to solve complex problems without requiring explicit domain-specific programming. It operates by representing candidate solutions as tree structures, which are iteratively refined using evolutionary operators such as selection, crossover, and mutation. These operators enable the exploration of a large solution space in search of optimal or near-optimal solutions.

In this study, GP is applied to the Student–Project Allocation problem by evolving candidate allocation solutions based on defined constraints, including student preferences, research domains, and supervisor capacity. The fitness of each candidate solution is evaluated using an objective function that measures the degree of alignment between students’ interests and allocated project topics. Through successive generations, the algorithm improves allocation quality by selecting and recombining high-performing solutions, thereby producing an optimized and efficient allocation framework.

Genetic Programming Algorithm, as defined by Spector *et al.* (2002), is as follows:

Step 1: Create an initial population of random individuals.

Step 2: Test the “fitness” of each individual.

Step 3: If an individual is sufficiently good, then stop and report success.

Step 4: Otherwise, create a new population from the fitter individuals, using “genetic operators” such as reproduction, mutation, and crossover.

Step 5: Replace the old population with the new population and return to Step 2.

Similarly, Jason (2011) introduced the pseudo-code of the Genetic Programming algorithm as follows:

```
Input: Population size, nodesfunc, nodesterm, Pcrossover, Pmutation,  
Preproduction, Palteration  
Output: Sbest  
  Population  $\leftarrow$  InitializePopulation(Populationsize, nodesfunc, nodesterm);  
  EvaluatePopulation(Population);  
  Sbest  $\leftarrow$  GetBestSolution(Population);  
  while  $\neg$ StopCondition() do  
    Children  $\leftarrow \emptyset$ ;  
    while Size(Children) < Populationsize do  
      Operator  $\leftarrow$  SelectGeneticOperator(Pcrossover, Pmutation,  
      Preproduction, Palteration);  
      if Operator  $\equiv$  CrossoverOperator then  
        Parent1, Parent2  $\leftarrow$  SelectParents(Population,  
        Population size);  
        Child1, Child2  $\leftarrow$  Crossover (Parent1, Parent2);  
        Children  $\leftarrow$  Child1;  
        Children  $\leftarrow$  Child2;  
      else if Operator  $\equiv$  MutationOperator then  
        Parent1  $\leftarrow$  SelectParents(Population, Populationsize);  
        Child1  $\leftarrow$  Mutate(Parent1);  
        Children  $\leftarrow$  Child1;  
      else if Operator  $\equiv$  ReproductionOperator then  
        Parent1  $\leftarrow$  SelectParents(Population, Populationsize);  
        Child1  $\leftarrow$  Reproduce(Parent1);  
        Children  $\leftarrow$  Child1;  
      else if Operator  $\equiv$  AlterationOperator then  
        Parent1  $\leftarrow$  SelectParents(Population, Populationsize);  
        Child1  $\leftarrow$  AlterArchitecture(Parent1);  
        Children  $\leftarrow$  Child1;  
      end  
    end  
  end  
  EvaluatePopulation(Children);
```

```
Sbest ← GetBestSolution(Children, Sbest);  
Population ← Children;  
End
```

Walker (2001) defines the fundamental elements of an individual as its genes, which come together to form code. An individual's program is a tree-like structure, and as such, there are two types of genes: *functions* and *terminals*. Terminals, in tree terminology, are leaves (nodes without branches), while functions are nodes with children. The function's children provide the arguments for the function.

Genetic Operators

Basically, there are three basic genetic operators in the genetic algorithm, as follows:

- Selection
- Cross Over
- Mutation

Selection

According to Jason (2011), selection is the first genetic operation in the reproductive phase of genetic programming. Its purpose is to choose the fittest individuals (programs) in the population that will create offspring for the next generation, commonly known as the mating pool. The mating pool thus selected takes part in further genetic operations, advancing the population to the next generation and hopefully closer to the optimal solution. Selection of individuals in the population is fitness-dependent and is done using different algorithms. Selection chooses more fit individuals in analogy to Darwin's theory of evolution – survival of the fittest. Too strong selection would lead to sub-optimal, highly fit individuals, and too weak selection may result in too slow evolution.

Cross over

The crossover operator is analogous to reproduction and biological crossover; more than one parent is selected, and one or more offspring are produced using the genetic materials of the parents. It is usually the first operator applied to the population. Chromosomes are selected from the population of parents to cross over and produce offspring. It is based on Darwin's theory of evolution, 'Survival of the Fittest'. Therefore, this operator is also known as reproduction. For each new solution to be produced, a pair of 'parent' solutions is selected for breeding from the pool selected previously.

Mutation

According to Jason (2011), Mutation can be defined as a genetic operator that can be used to maintain genetic diversity from one generation of a population of genetic algorithm chromosomes to the next. Mutation represents a change in the gene, its role is to provide a guarantee that the search algorithm is not trapped on a local optimum. The mutation operator flips a randomly selected gene in a chromosome. This probability is to be set low; if it is too high, the search will turn into a primitive random search. It typically ranges between 0.001 and 0.01.

4.3 System Organization / Operational Structure:

System Organization is the structure of how a system is set up. That structure defines how each division of a system is set up, the hierarchy of each individual module, how communication flows throughout the system, and the interaction between the modules.

The entire system will be structured using:

1. Use-case diagram,
2. Sequence diagram and

3. Activities diagram

4.3.1 Use-case Diagram

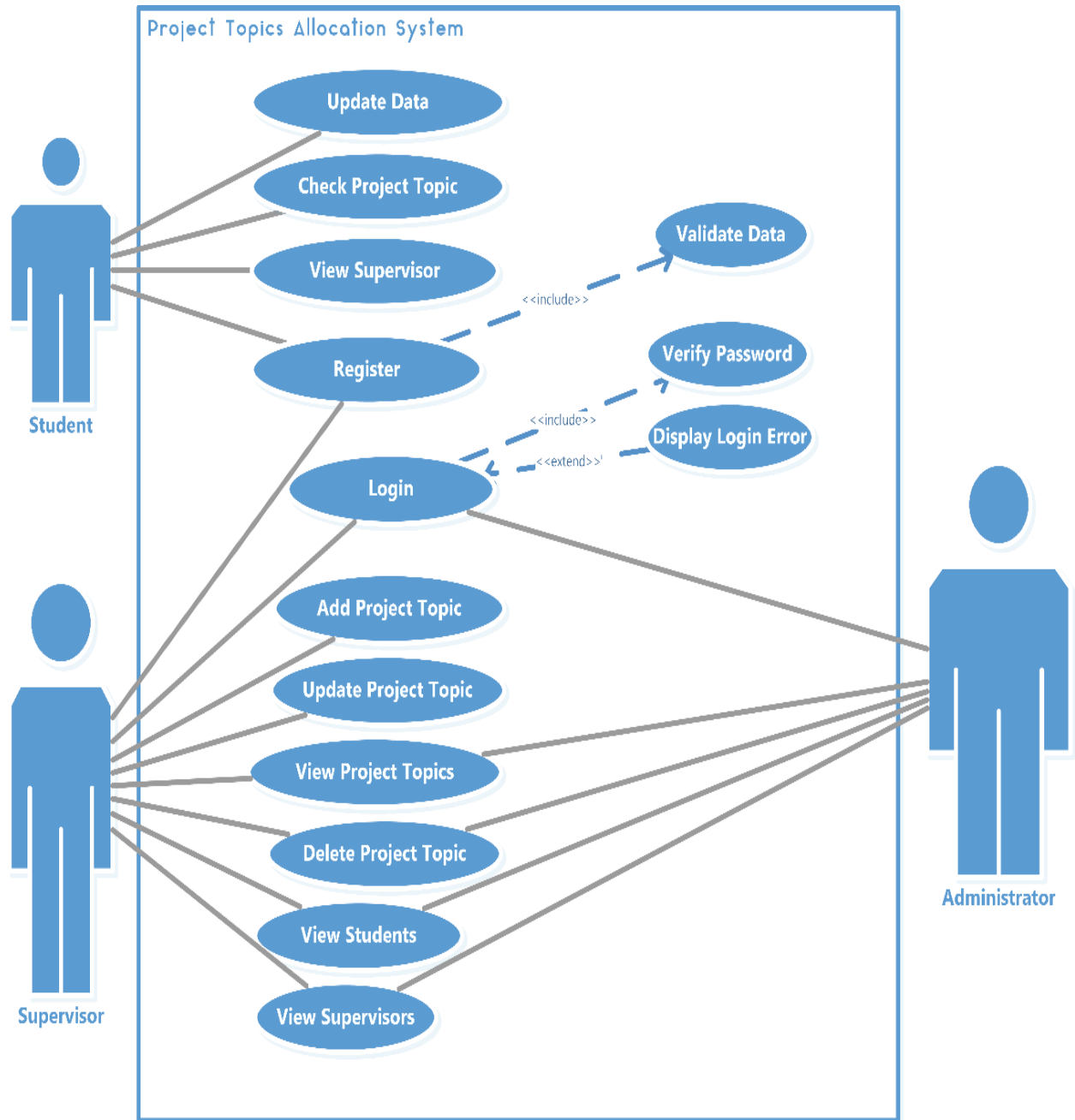


Figure 4.16: Use-case diagram of the proposed System

4.3.2 Sequence Diagram

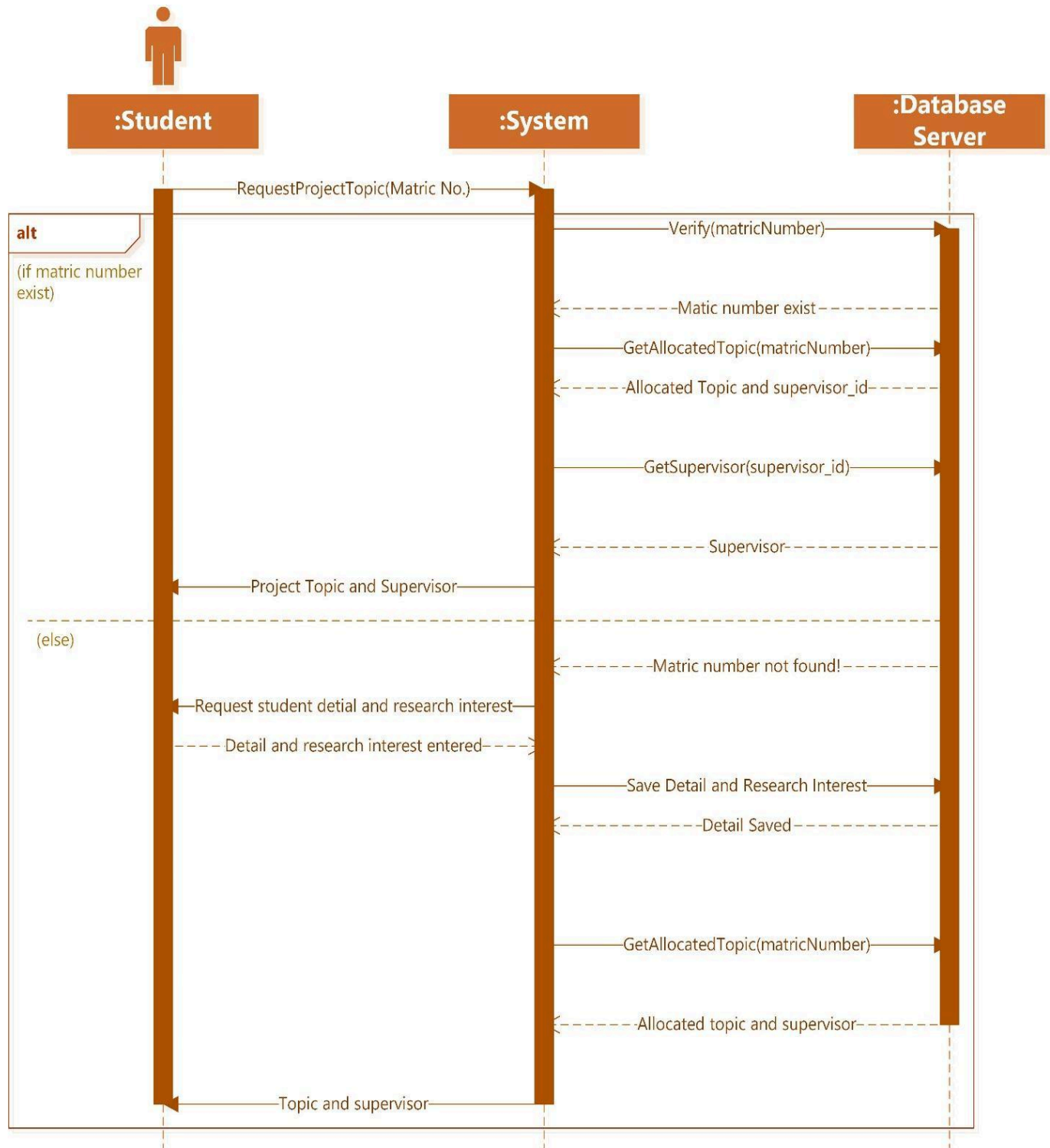


Figure 4.17: Sequence diagram of the proposed system

4.3.3 Activities Diagram

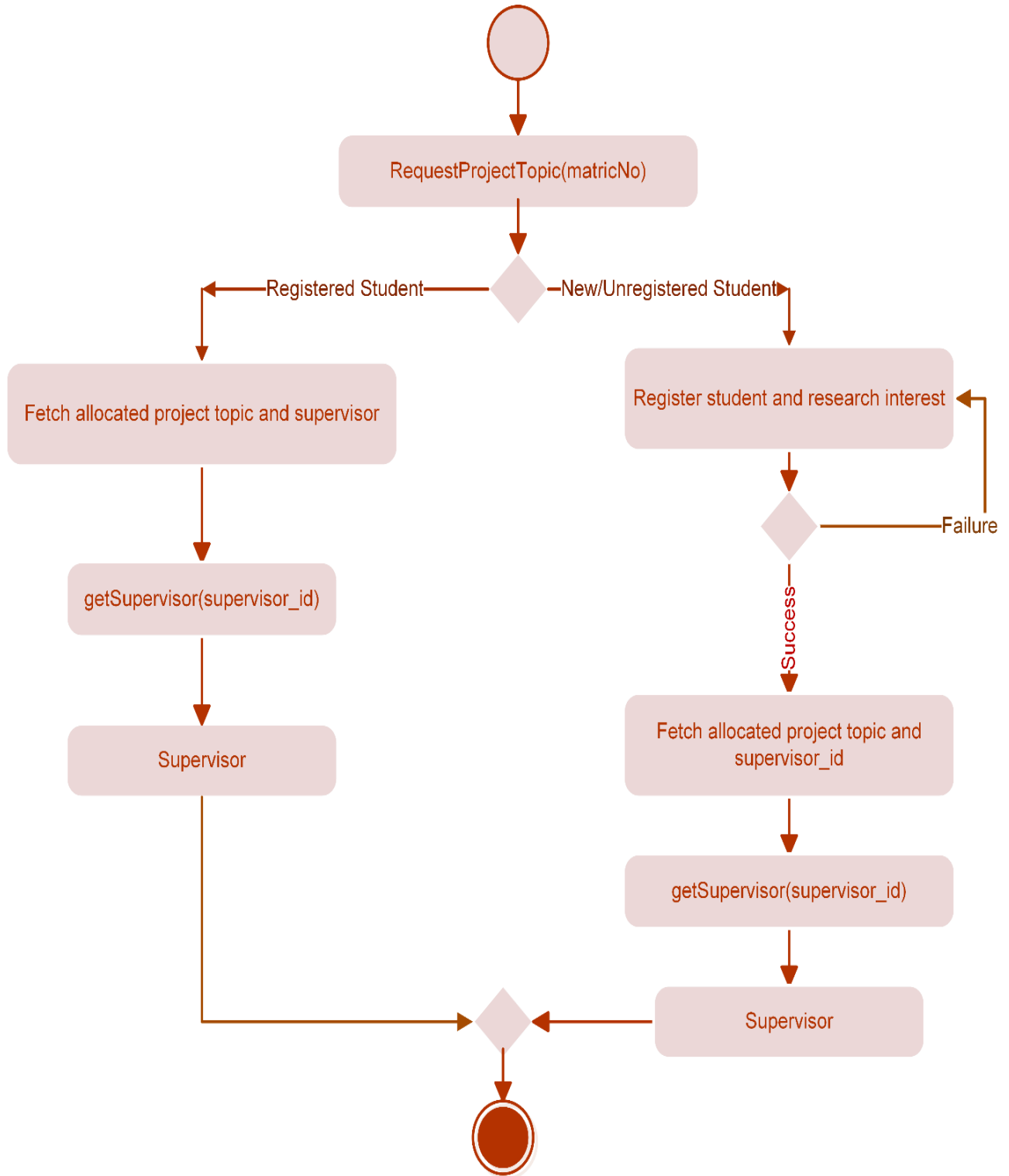


Figure 4.18: Activity diagram of the proposed system

CHAPTER FIVE

SYSTEMS IMPLEMENTATION

5.1 Implementation Approach

The implementation approach adopted for this study is Agile development. Agile methods are incremental and iterative development methods in which the increments are small and, typically, new releases of the system are created and made available to customers every two or three weeks. They involve customers in the development process to get rapid feedback on changing requirements. They minimize documentation by using informal communications rather than formal meetings with written documents (Sommerville, 2011).

The agile development model breaks the product into small incremental builds. These builds are provided in iterations. The Agile model believes that every project needs to be handled differently and the existing methods need to be tailored to best suit the project requirements. In agile, tasks are divided into time boxes, small timeframes to deliver specific features for a release.

According to Cohen *et al* (2004) and Sommerville (2011), agile methodologies include:

1. Extreme programming
2. Scrum
3. Lean development
4. Feature-driven development
5. Dynamic systems development methodology
6. Plan-driven and agile development

Specifically, this study adopted the feature-driven development model because it perfectly suits the purpose of this system. FDD comprises five basic processes, such as

developing an overall model, building the feature list, planning by feature, designing by feature, and building by feature.

The figure below is the block diagram of the Feature Driven Development Model.

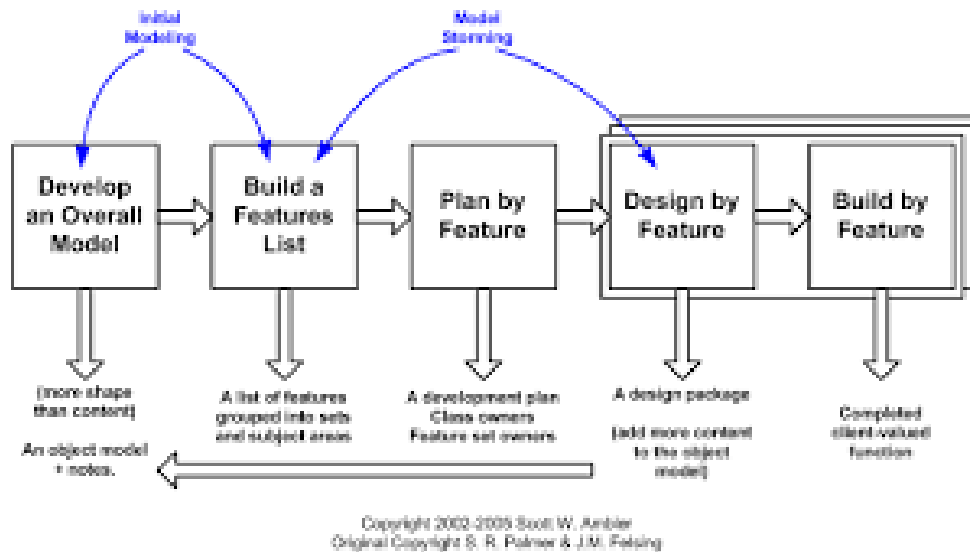


Figure 5.1: Feature-driven development model

Source: <http://agilemanifesto.org/Agile-manifesto-for-software-development>

5.3 Choice of Programming Language

PHP 7.2 was made the programming language of choice due to its New version of the Zend Engine, Improved performance (twice as fast as PHP 5.6), Significantly reduced memory usage, Abstract Syntax Tree, Consistent 64-bit support, Improved exception hierarchy, many fatal errors converted to exceptions, Secure random number generator, removed old and unsupported SAPIs and extensions, the null coalescing operator, Return and Scalar type declarations, Anonymous classes, Zero cost asserts. According to Branko Ajzele (2016), it has been more than two decades now since the birth of PHP. Created by Rasmus Lerdorf in 1994, the PHP acronym initially stood for *Personal Home Page*. Back then, PHP was merely a few *Common Gateway Interface (CGI)* programs in C, used to power a simple web page.

5.3.1 Development Environment and Tools

The following tools and environment are used in the development of the system:

- JetBrains PhpStorm 2018.2.2: This is used as the Integrated Development Environment (IDE) for coding and unit testing the system.
- PHPUnit is used for testing the various units and modules in the system during the coding stage.
- Apache WampServer64: This is the web server used to power the system for testing during development.
- MYSQL database is the Database Management System (DBMS) used for the development of the system. It is a high-end relational database management system.
- MYSQL Workbench 6.3CE was used for the modelling of the database schemas.
- Windows 10.1 was used as the development operating system.
- PHP 7.1.2 is the language of implementation used in the development of the system.
- Object-oriented programming was employed in the development of the system for easy separation of concerns and maintenance.

5.3.2 Application Programming Interface

Application Programming Interface, or API as it's commonly known, is a programming convention that allows for publicly exposed methods of an application to be accessed and manipulated outside of the program itself.

The following Application Programming Interfaces (APIs) were used in the development of the system: Symfony framework version 4.2 APIs, such as the routing, doctrine, security checker, profiler, debugger, HttpFoundation APIs, MySQL APIs, and PHP 7 core API.

Ajzele (2016) defines Application frameworks as a collection of functions, classes, configurations, and conventions all designed to support the development of web applications, services, and APIs. In a simple sentence, frameworks are here to make it easier and quicker for us to build our software. A great deal of programming languages out there have popular frameworks, and PHP is no exception to this.

Below are some general, high-level segments each modern framework should encompass, as outlined by Ajzele (2016). These account for:

- *Modularization*: It supports modular application development, allowing us to neatly and easily separate our code into functional building blocks (objects), whereas it is built in a modular manner.
- *Security*: It provides various cryptographic and other security tooling expected of a modern web application. Provides seamless support for things like authentication, authorization, and data encryption.
- *Extensiveness*: To easily adopt our application needs, allowing us to extend it according to our application needs and requirements.
- *Community*: It is actively developed and supported by a vibrant and active community.
- *Performance*: Built with high performance in mind.
- *Enterprise Ready*: Depending on the type of project at hand, most likely we would want to target a framework that flags itself as enterprise-ready. Making us confident enough to run critical and high-usage business applications on top of it.

This study employs Symfony Framework version 4.2 for the implementation of the study. Symfony is released under an MIT license, and can be downloaded from <http://symfony.com>. Symfony is such a popular framework due to its interoperability.

According to Ajzele (2016), the idea of "Don't lock yourself up within Symfony!" made it popular among developers as it allowed for building applications that precisely meet our needs. By embracing the "don't reinvent the wheel" philosophy, Symfony itself makes heavy use of existing PHP open-source projects as part of the framework, including:

- Doctrine (or Propel): Object-relational mapping layer
- PDO database abstraction layer (Doctrine or Propel)
- PHPUnit: A unit testing framework
- Twig: A template engine
- Swift Mailer: An e-mail library

Depending on our project needs, we can choose to use a full-stack Symfony framework, or the Silex micro-framework, or simply some of the components individually. Out of the box, Symfony provides a lot of structural ground for new web applications. It does so via its bundle system. Bundles are sort of like micro-applications inside our main application. Within them, the entire app is nicely structured into models, controllers, templates, configuration files, and other building blocks. Being able to fully separate logic from different domains helps us to keep a clean separation of concerns and autonomously develop every single feature of our domain. Symfony is one of the PHP pioneers when it comes to embracing dependency injection across the framework, allowing it to achieve decoupled components and to keep high flexibility of code. Documented, modular, highly flexible, performant, supported, those are the attributes that make Symfony a choice worth evaluating.

Below are various individual Symfony framework features essential for building a modular application as described by Ajzele (2016):

- Controller

- Routing
- Templates
- Forms
- The bundle system
- Databases and Doctrine
- Testing
- Validation

Below is an architectural view of the symphony framework

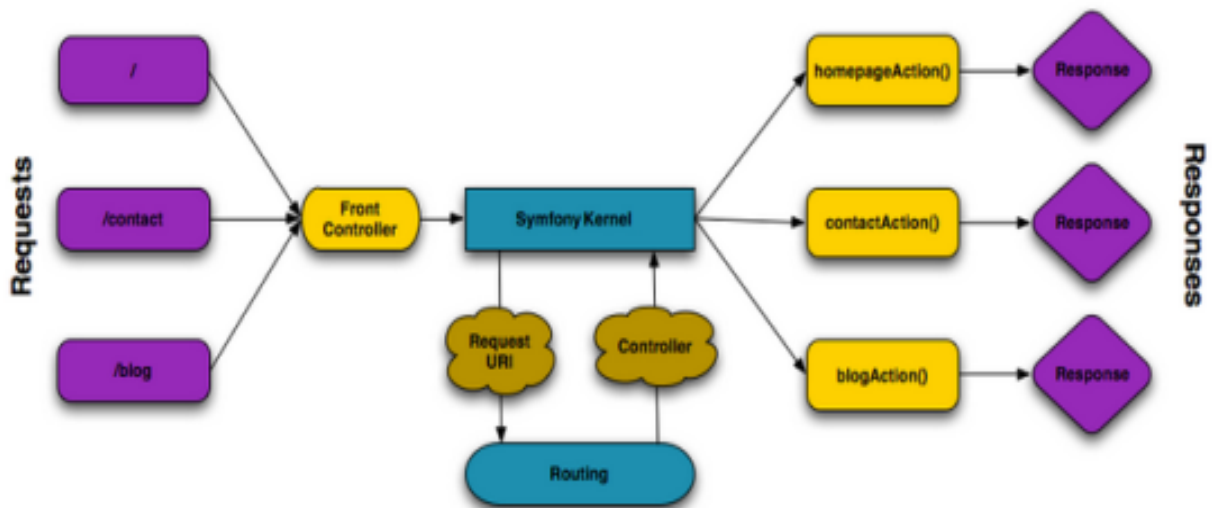


Figure 5.2: Symfony Architecture

Source: <http://symfony.com/documentation/component/httpfoundations>

5.4 Software and Hardware Requirement Tools

The minimum software and hardware requirements for the developed system to operate at its optimality are as follows:

Server Requirement

Operating System	Linux
------------------	-------

PHP Engine	Apache Zend
MySQL	1GB
Hard disk	1GB
Bandwidth	6Gps
Ram	1GHz
Processor	1GHz

Client Requirement

Operating System	All
Software	Web Browser
Protocol	HTTPv1.1
Javascript	Enable
Screen Resolution	900x600px
Bandwidth	50MBps
Cookies	Enable

5.5 Testing

Types of testing adopted in this stud includes:

1. Unit Testing
2. Integration Testing
3. System Testing

5.5.1 Unit Testing

Unit testing focuses on testing individual modules of the system. It is a software testing method by which individual units of source code, usage procedures, and operating procedures

are tested to determine whether they are fit for use. For example, unit testing was carried out on each function, such as the modules that perform mutation, selection, crossover, and allocation in the developed system. PHPUnit is used for the unit testing of the system.

5.5.2 Integration Testing

After unit testing has been done with satisfaction for each component, the integration testing is started to ensure there is proper interaction between the system modules, that is, there is no conflict with the communication between one module to another, for example, the interaction between the logs module and the main interface module, integration testing ensures that there is no conflict in the data declaration in different modules. Furthermore, an integration test was carried out between the selection, crossover, and mutation functions for our developed system to achieve its aim.

5.5.3 System Testing

At this level of testing, the whole software system was tested so as to certify its conformity with the design specification and requirements.

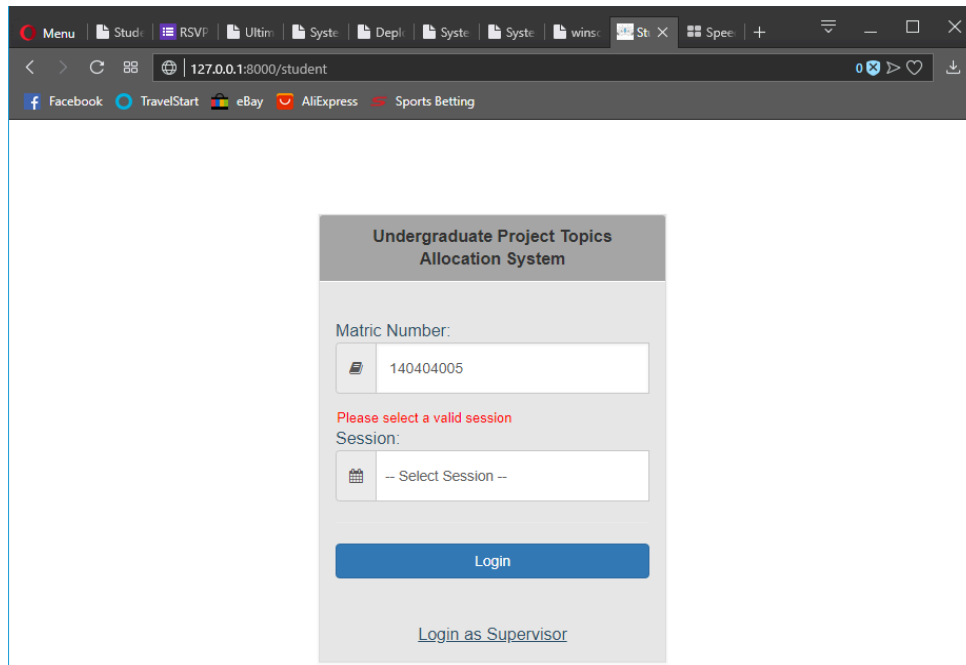


Figure 5.3: Login Testing Interface

5.6 Deployment and Operation of the System

The details of the general mode of operating the system will be made known to the user through the delivery of the user's documentation (manual). As it is known, the interface and output designs, which are customarily and user-friendly designed, are mainly meant for the end-user for accessing and viewing, while the input designs (files) are to be maintained by the system user. The user's documentation will therefore be a descriptor of the use of the various files and input form designs, as well as the whole system in general.

Operating the System

The system allows students and supervisors to provide details used for registration and allocation which include but not limited to username, first name, middle name, last name, email, phone number, project topics (supervisors only), three research interest (students only), the user can click on the register button for new registration and the output below will be shown.

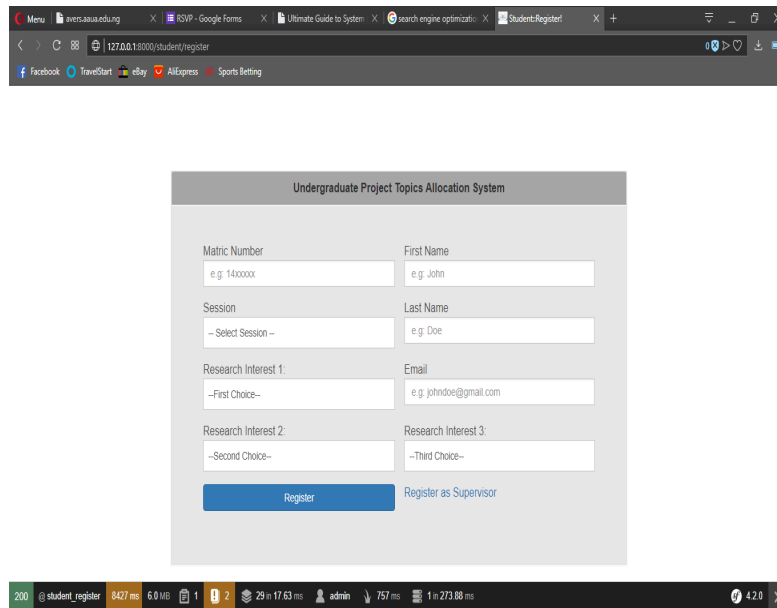


Figure 5.4: Student Registration Interface

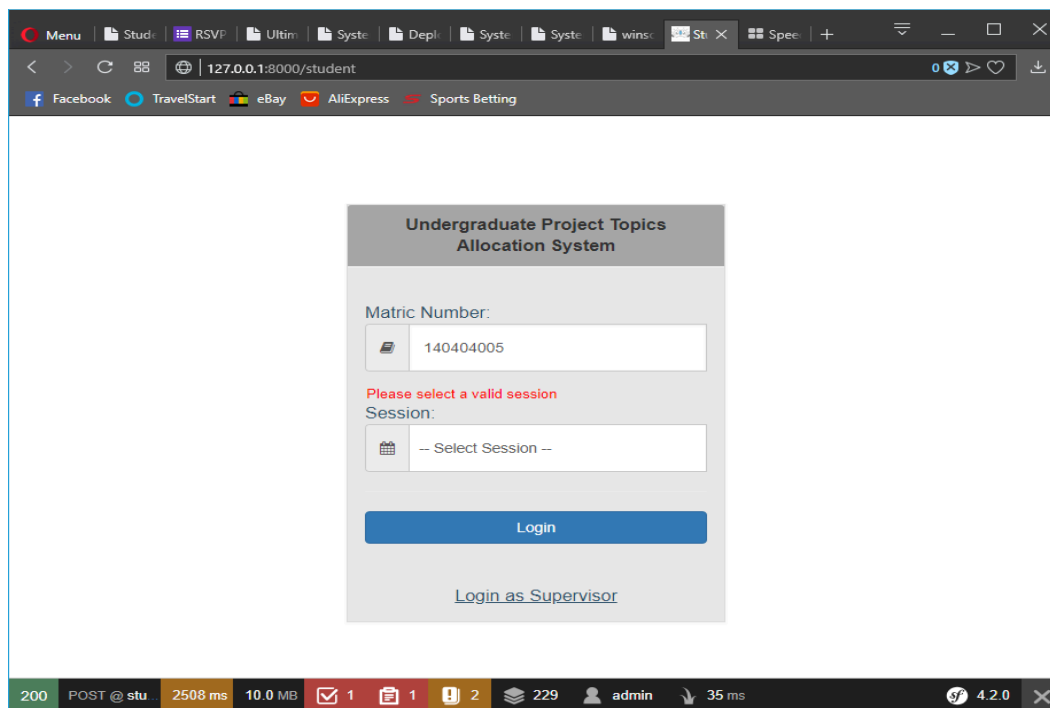


Figure 5.5: Student Login Interface

Once the student has signed up successfully and logged in, the student is granted access to the dashboard of the system to be able to view the allocated project topic and update details.

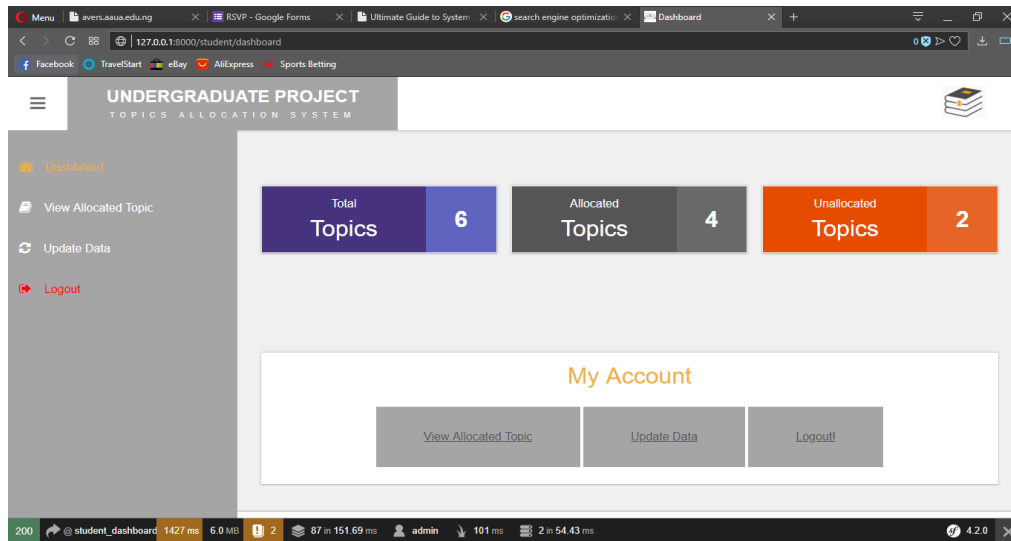


Figure 5.6: Student Dashboard Interface

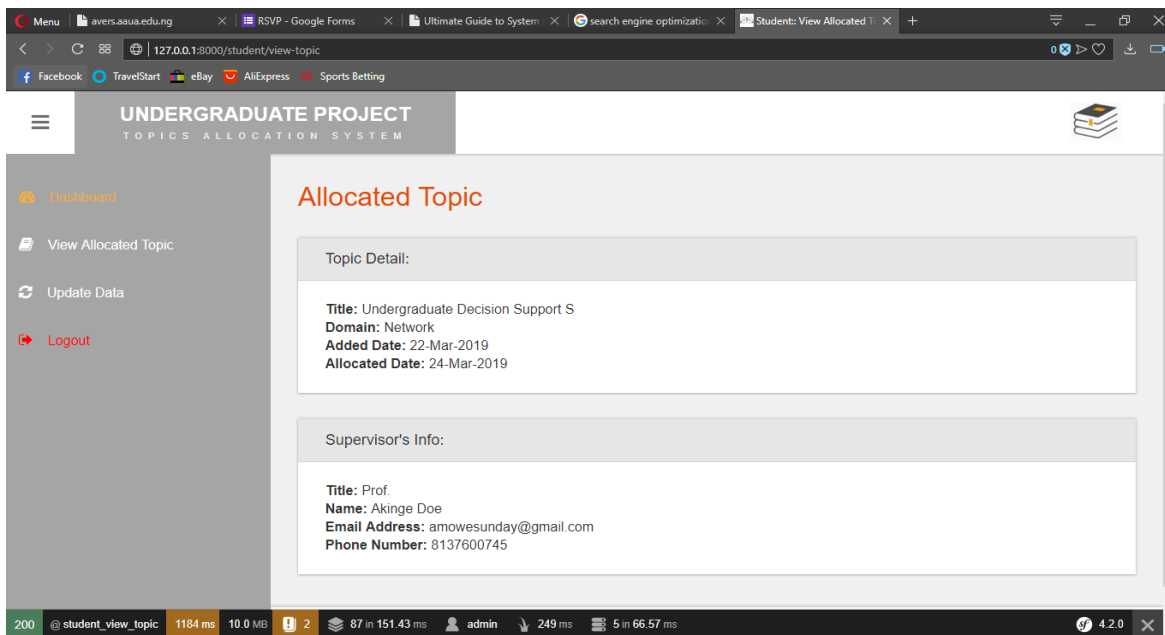
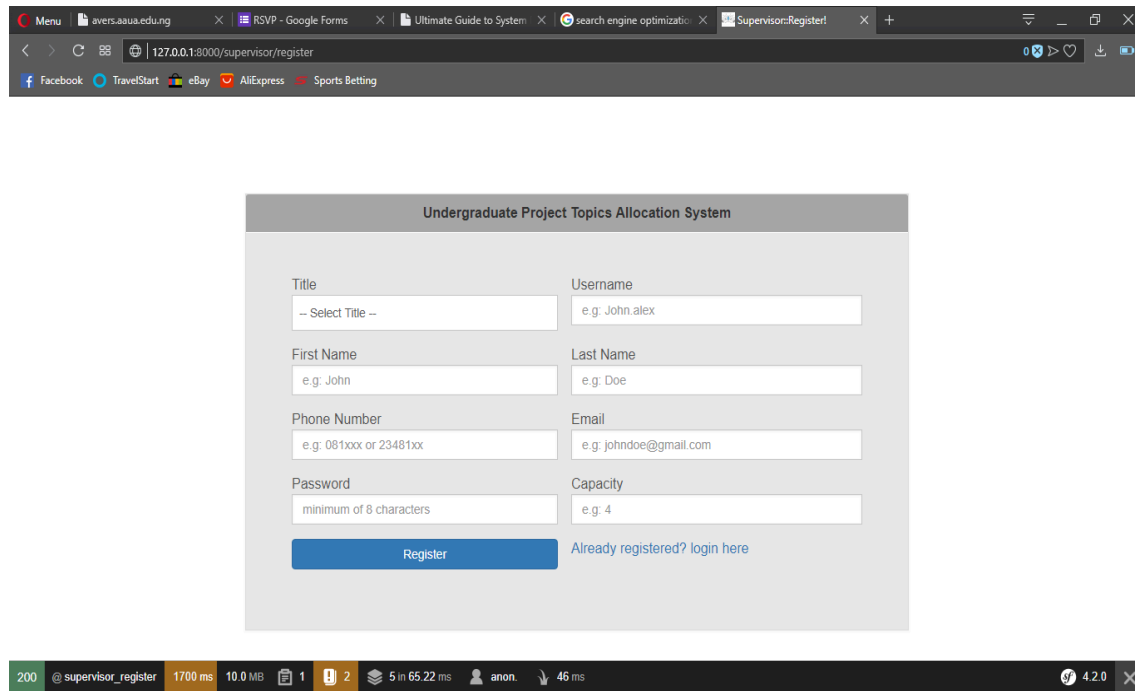


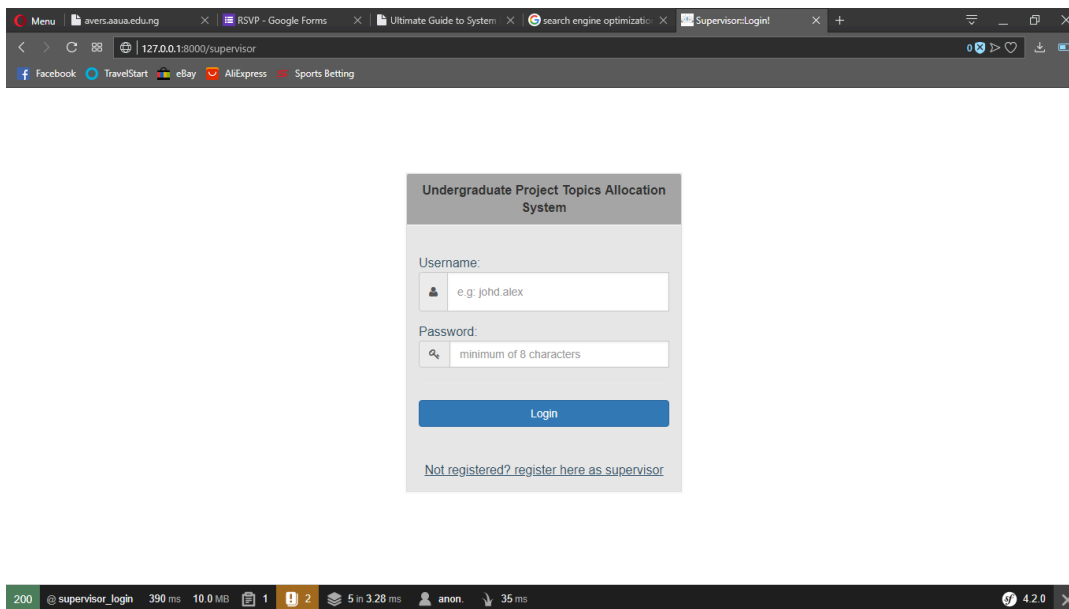
Figure 5.7: Student Allocated Project Topic Interface

The registration output shown below accepts users' details and send it to the backend for validation and storage, and also to be able to grant access to supervisors when logging in.



The screenshot shows a web browser window with the URL `127.0.0.1:8000/supervisor/register`. The page displays the "Undergraduate Project Topics Allocation System" registration form. The form includes fields for Title (a dropdown menu), Username, First Name, Last Name, Phone Number, Email, Password, and Capacity. A "Register" button is at the bottom left, and a link "Already registered? login here" is at the bottom right. The browser's taskbar at the bottom shows the process `@supervisor_register` running.

Figure 5.8: Supervisor Registration Interface



The screenshot shows a web browser window with the URL `127.0.0.1:8000/supervisor`. The page displays the "Undergraduate Project Topics Allocation System" login form. The form includes fields for Username and Password. A "Login" button is at the bottom. Below the button is a link "Not registered? register here as supervisor". The browser's taskbar at the bottom shows the process `@supervisor_login` running.

Figure 5.9: Supervisor login Interface

Once the supervisor has signed up successfully and logged in, the supervisor is granted access to the dashboard of the system to be able to add a project topic and update details.

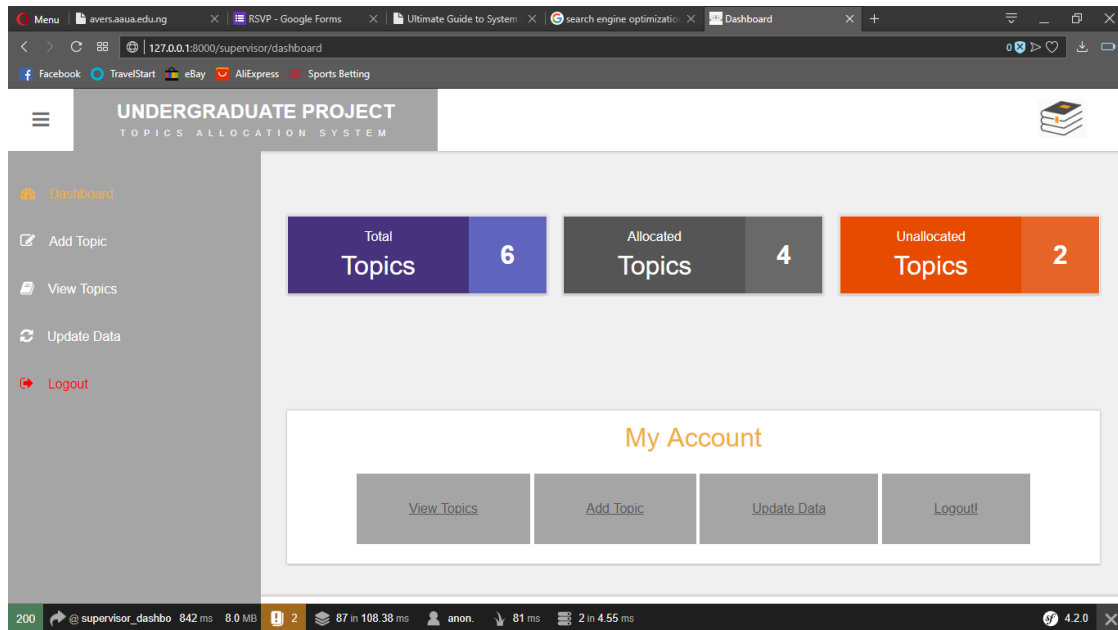


Figure 5.10: Supervisor's Dashboard Interface

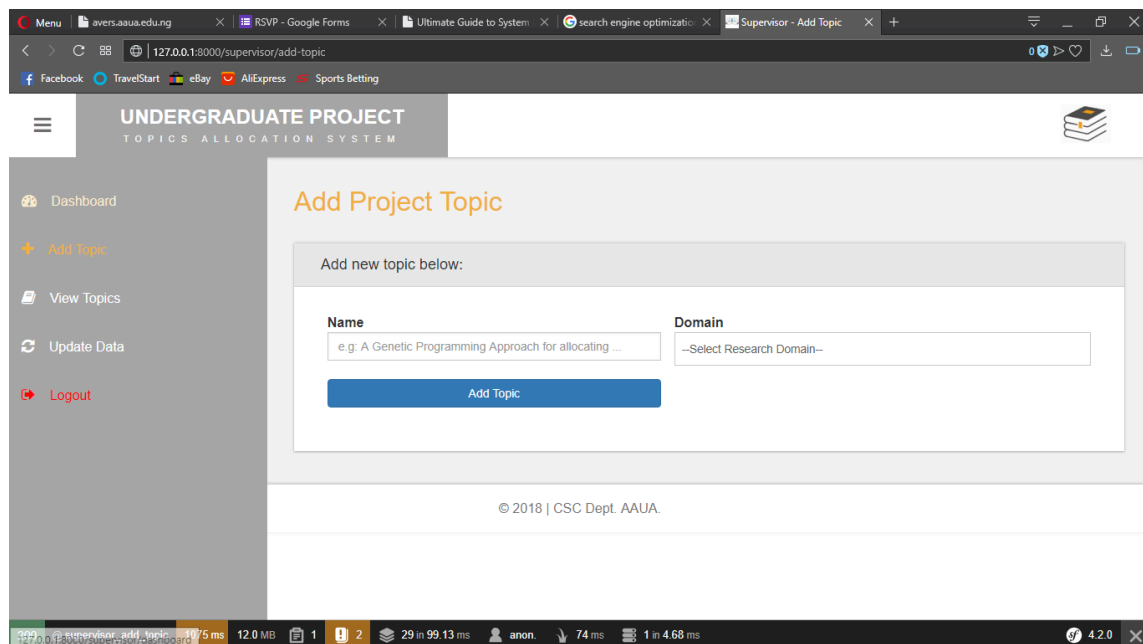


Figure 5.11: Supervisor's Add Project Topic Interface

5.7 Documentation

Documents describe the product at all levels of development, including the finished product.

An Undergraduate Project Topics Allocation System Documentation

On loading/visiting the homepage (landing page) of the system, the following interface pops up, which displays the total number of project topics, allocated project topics, and unallocated project topics.



Figure 5.12: Landing page Interface

1. Index Icon: This icon, when clicked, redirects the user to the landing page of the system.
2. Project Topics menu: This menu, when clicked, redirects the user to the list of project topics, indicating allocated and unallocated.

3. Overview Menu: This menu redirects the user to the system overview page, showing the system details.
4. Contact us Menu: The contact us menu redirects the user to the contact us form, which is placed at the bottom of the page.
5. Login Menu: The login menu relocates the user to the login interface, where students can log in to the system.
6. Register Menu: The register menu, when clicked, redirects the user to the registration page.

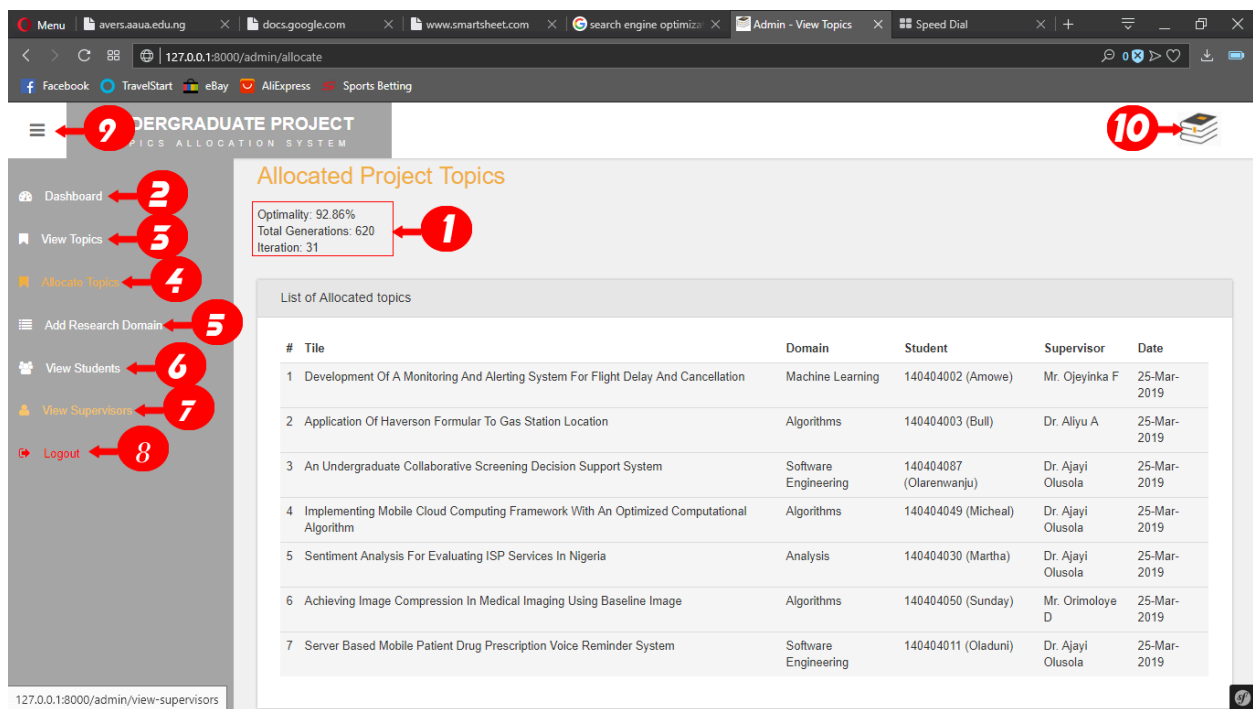


Figure 5.13: Allocate project topics interface (admin only)

1. Allocation Statistics Pane: This pane displays the allocation statistics, such as the optimization level, total generations, and iteration count.

2. Dashboard menu: This menu redirects the user to the admin dashboard interface, where the user can view a list of allocated and unallocated project topics.
3. View topics menu: This menu redirects the user to the admin view topics interface, where the admin can view a list of research topics.
4. Allocate topics menu: This menu redirects the admin to the allocate topics interface, where the admin can view the list of allocated topics, and if there are no allocated topics, the admin can click on the allocate button for the system to allocate project topics to students using a genetic programming approach for optimized allocation.
5. Add research domain menu: This menu, when clicked, redirects the admin to the add research domain interface, where the admin can add a new research domain.
6. View student menu: The view student menu, when clicked, relocates the admin to the view student interface, where the admin can view the list of registered students.
7. View supervisor menu: This menu, when clicked, redirects the admin to the view supervisor interface, where the admin can view the list of registered supervisors.
8. Logout menu: This menu, when clicked, logs out the current logged-in admin and redirects the user to the admin login interface.
9. Hide/show ribbon: This ribbon-like menu, when clicked, hides and shows the menu pane of the system.
10. Index/home icon: This icon redirects the admin to the landing page of the system when clicked.

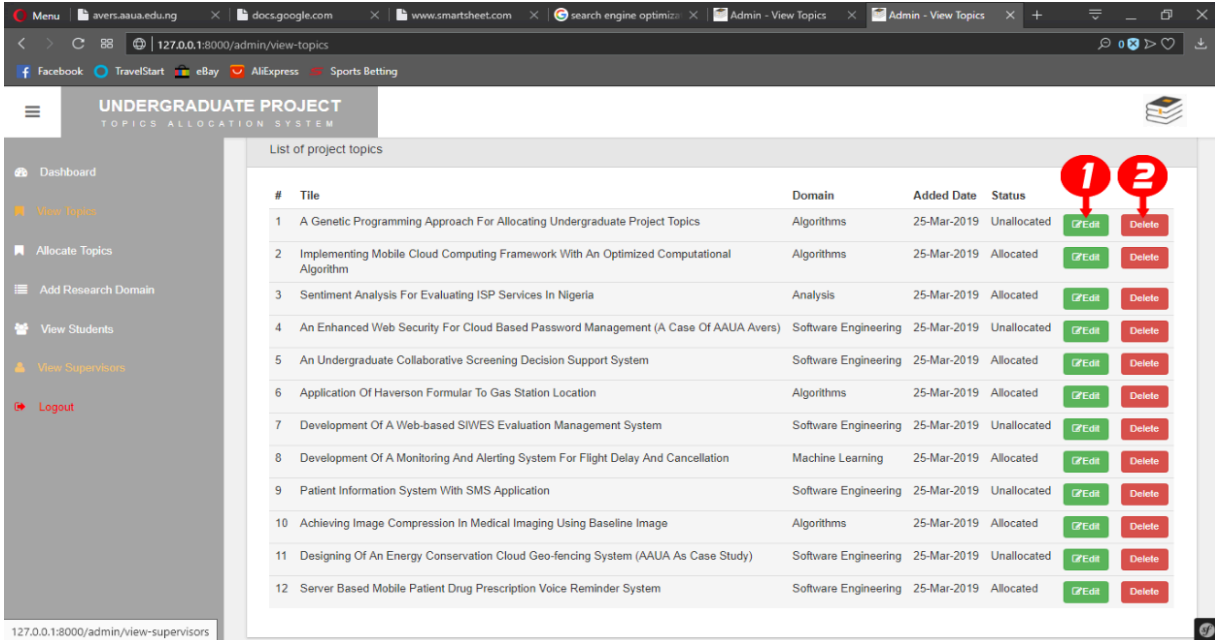


Figure 5.14: View List of topics interface

1. Edit Button: This button, when clicked, redirects the admin to the edit topic interface, where the admin can view the details of a particular project topic.
2. Delete Button: This button, when clicked, removes a particular project topic from the database.

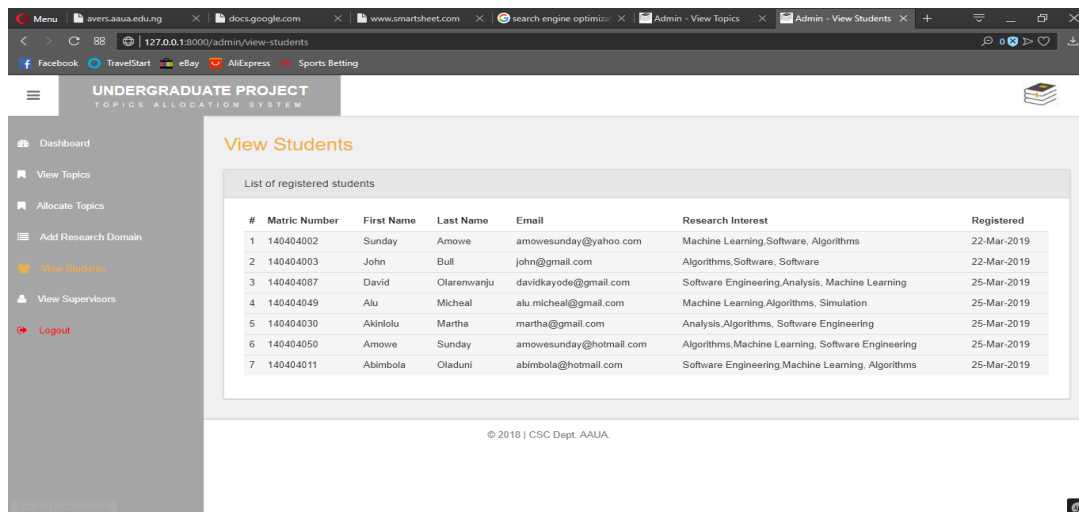


Figure 5.15: View allocated topics interface

CHAPTER SIX

SUMMARY, CONCLUSION, AND RECOMMENDATION

6.1 SUMMARY

Undergraduate project allocation refers to the systematic distribution of project topics to students in order to facilitate effective supervision and academic oversight. While the assignment of supervisors is not inherently challenging in institutions with well-defined policies, inconsistencies may arise in their absence.

In this context, Student–Project Allocation (SPA) is conceptualized as a two-sided matching problem that involves assigning students to projects under preference and capacity constraints. However, the effectiveness of this process largely depends on the compatibility between students’ research interests and supervisors’ areas of expertise. Consequently, achieving an optimal and stable match remains a significant challenge, particularly in environments characterized by diverse preferences and limited supervisory capacity.

6.2 FINDINGS

The following are the findings from the study:

1. The results of the findings show that Genetic Programming is a more optimized approach for allocating undergraduate project topics compared to stable marriage, rule-based, and matching approaches, given an ideal circumstance.
2. The research and output evaluated show that Genetic programming is resource-intensive in terms of computational time and memory.
3. The research exposes the power of modularity in the Symphony 4.0 framework and the computational speed of PHP 7.2+

6.3 CONTRIBUTIONS TO KNOWLEDGE

The study established:

1. A Genetic Programming approach for increased efficiency and effectiveness in undergraduate project topics allocation to improve the rate and speed at which project topics are systematically allocated to undergraduate students.
2. A system with an optimized undergraduate project topics allocation from a given non-empty set of project topics using genetic programming.

6.4 CONCLUSION

The manual method of project topic allocation in higher institutions is in need of reconsideration, as project writing is a mandatory and sensitive academic requirement for the completion of undergraduate degree programmes. Evidence from sampled documents in the case study indicates that project allocation is commonly carried out either through a sequential approach, where students' matriculation numbers are matched with academic staff/lecturer IDs, or, more frequently, based on lecturers' seniority.

However, this manual approach is characterized by several limitations, including favoritism (godfatherism), inefficient and ineffective allocation and supervision processes, redundancy in project topics, and incomplete project execution. Consequently, this study proposes a model that provides an improved mechanism for allocating project topics to undergraduate students based on their research interests and academic domains.

Empirical evaluation demonstrates that the Genetic Programming-based algorithm outperforms the traditional manual method, achieving an allocation optimality score of 85.86%. This result underscores the effectiveness of evolutionary computation techniques in addressing complex combinatorial assignment problems within academic environments. Consequently, the proposed approach offers a scalable and more equitable alternative for project topic allocation in higher education institutions.

6.5 DIRECTIONS FOR FURTHER STUDIES

The developed system presented in this study is designed for the allocation of undergraduate project topics, defined as the systematic distribution of project topics to students to enable effective monitoring and supervision based on multiple criteria, including research interest and academic domain alignment. However, other quality criteria, such as related project topics allocation, such as supervisor's capacity, and postgraduate allocation, can also be considered in future research work in the allocation of project topics.

REFERENCES

- Abraham, D. J., Irving, R. W., & Manlove, D. F. (2007). Two algorithms for the Student-Project Allocation problem. *Journal of Discrete Algorithms*, 5(1), 73–90. <https://doi.org/10.1016/j.jda.2006.03.006>
- Bernd Bruegge, & Dutoit, A. H. (2010). *Object-oriented software engineering : using UML, patterns, and Java*. Prentice Hall.
- Brownlee, J. (2012). *Clever algorithms : nature-inspired programming recipes*. Lulu.com.
- Fabiyi, A. A., Ramoni, T. A., & Adetutu , A. O. (2016). Development of Student Project Allocation System Using Matching Algorithm. *International Conference of Science, Engineering & Environmental Technology (ICONSEET)*, 1(22), 153–160.
- Gupta, P., & Kulkarni, N. (2013). An Introduction of Soft Computing Approach over Hard Computing. *International Journal of Latest Trends in Engineering and Technology (IJLTET)*, 3(1), 254–258. <https://www.ijltet.org/wp-content/uploads/2013/09/36.pdf>
- Ibrahim, D. (2016). An Overview of Soft Computing. *Procedia Computer Science*, 102(1877-0509), 34–38. <https://doi.org/10.1016/j.procs.2016.09.366>. 12th International Conference on Application of Fuzzy Systems and Soft Computing, ICAFS 2016, 29-30 August 2016, Vienna, Austria.
- Iwama, K., Miyazaki, S., & Yanagisawa, H. (2012). Improved approximation bounds for the Student-Project Allocation problem with preferences over projects. *Journal of Discrete Algorithms*, 13, 59–66. <https://doi.org/10.1016/j.jda.2012.02.001>
- Kwanashie, A., Irving, R. W., Manlove, D. F., & Colin. (2015). Profile-Based Optimal Matchings in the Student/Project Allocation Problem. *Lecture Notes in Computer Science*, 213–225. https://doi.org/10.1007/978-3-319-19315-1_19
- Lei, W., & Chim, T. W. (2017). *A Web-based Project Allocation System* [Thesis]. A paper presented to the University of Hong Kong, January 12, 2017.

- Manlove, D. F., & O'Malley, G. (2008). Student-Project Allocation with preferences over Projects. *Journal of Discrete Algorithms*, 6(4), 553–560. <https://doi.org/10.1016/j.jda.2008.07.003>
- Moussa, M. I., & El-Atta, A. H. A. (2011). A Visual Implementation of Student Project Allocation. *International Journal of Computer Theory and Engineering*, 3(2), 178–184. <https://doi.org/10.7763/ijcte.2011.v3.302>
- Mutula, S. M. (2009). *Building Trust in Supervisor-Supervisee Relationship: Case Study of East and Southern Africa* [Thesis]. Paper Presented at the Progress in Library and Information Science in Southern Africa (PROLISSA) Conference at the University of South Africa (UNISA), March 4-6, 2009.
- Pan, L., Chu, S., Han, G., & Huang, J. (2009). *Multi-criteria Student Project Allocation: A Case Study of Goal Programming Formulation with DSS...* ResearchGate. https://www.researchgate.net/publication/260386845_Multi-criteria_Student_Project_Allocation_A_Case_Study_of_Goal_Programming_Formulation_with_DSS_Implementation
- Salami, H., & Mamman, E. (2016). A Genetic Algorithm for Allocating Project Supervisors to Students. *International Journal of Intelligent Systems and Applications (IJISA)*, 8(10), 51–59. <https://doi.org/10.5815/ijisa.2016.10.06>
- Sam, L., & Ian, P.-H. (2016). *Solving The Student Project Allocation Problem* [Thesis].
- Sommerville, I. (2011). *Software Engineering* (9th ed.). Pearson.
- Spector, L., Langdon, W. B., O'Reilly, U.-M., & Angeline, P. J. (1999). *Advances in genetic programming*. (Vol. 3, pp. 91–450). The MIT Press 1999.
- Walker, M. (2001). *Introduction to Genetic Programming*. https://icog-labs.com/wp-content/uploads/2014/07/Introduction_to_GP_Matthew-Walker.pdf

APPENDICES

APPENDIX I (Material Used / Data Used / Instrument Used)

1. Jetbrians PHPStorm 2019
2. MySQL v5.7
3. Microsoft Visual Studio (2016)
4. Adobe Fireworks C6
5. WampServer64 v3.1
6. Windows Snipping tool
7. HeidiSQL 9.5
8. WinSCP
9. Opera Mini 58.0.3135.107

APPENDIX II (Source Listing – Codes)

1. **WorkerThread.php:** This class triggers the execution of the Genetic Programming that does the project topics allocation.

```
$this->interestBoard = [];  
$topicIDs = [];  
foreach ($this->topics as $topic){  
    $priority = [];  
    foreach ($this->students as $student){  
        if($student->getFirstInterest() ==  
$topic->getDomain()->getName()){  
            $priority[$student->getStudentId()] = 1;  
        }else if($student->getSecondInterest() ==  
$topic->getDomain()->getName()){  
            $priority[$student->getStudentId()] = 0.5;  
        }else if($student->getSecondInterest() ==  
$topic->getDomain()->getName()) {  
            $priority[$student->getStudentId()] = 0.25;  
        }else{  
            $priority[$student->getStudentId()] = 0;  
        }  
    }  
    $this->interestBoard[$topic->getTopicId()] = $priority;  
    $topicIDs[] = $topic->getTopicId();  
}  
$population = new Population();  
while($population->size() < 20){  
    $population->add($this->initialDNA($this->students,$topicIDs));  
}  
$generation = new Generation($population);  
//compute optimize solution.  
while($this->fitness < 90){  
    $this->loop++;  
    //compute fitness..  
    $generation->fitness(function(DNA $dna){  
        $fitness = 0;  
        foreach($dna->getGenes() as $studentID => $topicID){  
            $fitness += $this->interestBoard[$topicID][$studentID];  
        }  
        if($this->fitness < (($fitness/$dna->size()) * 100)){  
            $this->fitness = ($fitness/$dna->size()) * 100;  
            $this->bestFit = $dna;  
        }  
        return $fitness;  
    });  
    //perform Natural Selection  
    $generation->naturalSelection();
```

```

        //Compute mutation..
        $generation = $generation→next($this→interestBoard, function (DNA
$dna){
            $studentID = array_keys($dna→getGenes());
            $studentID = $studentID[array_rand($studentID)];
            $topicID = array_values($dna→getGenes());
            $value =
array_values(array_diff(array_keys($this→interestBoard),$topicID));
            $value = $value[array_rand($value)];
            $dna→set($studentID,$value);
            $this→generationCount++;
            return $dna;
        });
    }
    //Optimized allocation
    return $this→bestFit;

```

2. **Generation.php:** This class performs the population generation phase of genetic programming, where a new population (set of DNAs) with better fitness is generated.

```

if(count($this→matingPool) < 1){
    return new Generation(new Population());
}

$population = new Population();
while($population→size() < $this→population→size()) {
    $parent1 = $this→matingPool[array_rand($this→matingPool)];
    $parent2 = $this→matingPool[array_rand($this→matingPool)];
    $child = $this→crossover($parent1, $parent2, $scoreBoard);
    $population→add($this→mutate($child, $callback));
}
return new Generation($population);

/**
 *This method performs crossover on the given parent datas.
 * @param DNA $parent1
 * @param DNA $parent2
 * @param array $scoreBoard
 * @return DNA
 */
public function crossover(DNA $parent1, DNA $parent2, array
$scoreBoard): DNA {
    $child = new DNA();
    //swap
    foreach($parent1→getGenes() as $studentID ⇒ $topicID){

```



```

        if(($scoreBoard[$topicID][$studentID] <
$scoreBoard[$parent2→get($studentID)][$studentID]) &&
!in_array($parent2→get($studentID),array_values($child→getGenes()))){
            $child→set($studentID,$parent2→get($studentID));
        }else{
            $newvalue =
array_diff(array_values($parent1→getGenes()),array_values($child→getGe
nes()));
            $child→set($studentID,$newvalue[array_rand($newvalue)]);
        }
    }
    return $child;
}

/**
 *This returns an array of the mating pool
 */
public function getMatingPool(): array {
    return $this→matingPool;
}

/**
 * This method returns the fitness of the generation.
 * @param callable $callable
 */
public function fitness(callable $callable): void{
    $this→fitness = array_map($callable,
$this→population→getDnas());
}

/**
 * @return array
 */
public function getFitness(): array {
    return $this→fitness;
}

/**
 * @return Population
 */
public function getPopulation(): Population{
    return $this→population;
}

/**
 * @param Population $population
 */
public function setPopulation(Population $population): void{

```

```

        $this->population = $population;}

/**
 * This method performs natural selection by
 * creating a mating pool with the DNA with
 * highest fitness as the highest occurrence
 */
public function naturalSelection(){
    $this->matingPool = [];
    $totalFitness = array_sum($this->fitness) <
1?1:array_sum($this->fitness);
    foreach($this->fitness as $key => $value){
        $pbSize = ($value/$totalFitness) * $this->population->size();
//This is the probability size.
        for($i = 0; $i < round($pbSize); $i++){
            $this->matingPool[] = $this->population->get($key);
        }
    }
}

```

3. **Population.php:** This class contains the individual population and the necessary actions

```

/**
 * This is Population class.
 * User: MaHome
 * Date: 2/27/2019
 * Time: 2:12 PM
 */
class Population{
    //Class properties goes here.....
    /**
     * @var array
     */
    private $dnas;

    public function __construct(array $dnas = []){
        $this->dnas = $dnas;
    }

    /**
     * @return int
     */
    public function size():int {
        return count($this->dnas);
    }
}

```

```

/**
 * @param array $dnas
 */
public function setDnas(array $dnas): void{
    $this->dnas = $dnas;
}

/**
 * @return DNA[]
 */
public function getDnas(): array{
    return $this->dnas;
}

/**
 * @param DNA $dna
 */
public function add(DNA $dna){
    $this->dnas[] = $dna;
}

/**
 * @param int $index
 * @return DNA
 */
public function get(int $index): DNA{
    return $this->dnas[$index];
}
}

```

4. DNA.php: This class contains set of genes and the necessary actions required

```

class DNA{
    //Class properties goes here.....
    /**
     * @var array
     */
    private $genes;

    /**
     * This is the class constructor.
     * DNA constructor.
     * @param array/null $genes
     */

```

```

public function __construct(array $genes = []){
    $this->genes = $genes;
}

/**
 * @return array
 */

public function getGenes(): array{
    return $this->genes;
}

/**
 * @param array $genes
 */
public function setGenes(array $genes): void{
    $this->genes = $genes;
}

/**
 * This method change the gene value of a given index
 * @param int $index
 * @param $value
 */
public function set(int $index, $value){
    $this->genes[$index] = $value;
}

/**
 * This method add a new gene information into the genes
 * @param $value
 */
public function add($value){
    $this->genes[] = $value;
}

/**
 * This method returns the gene in a given index
 * @param int $index
 * @return mixed
 */
public function get(int $index){
    return $this->genes[$index];
}

/**
 * This returns the number of genes present.
 * @return int
 */
public function size():int {

```

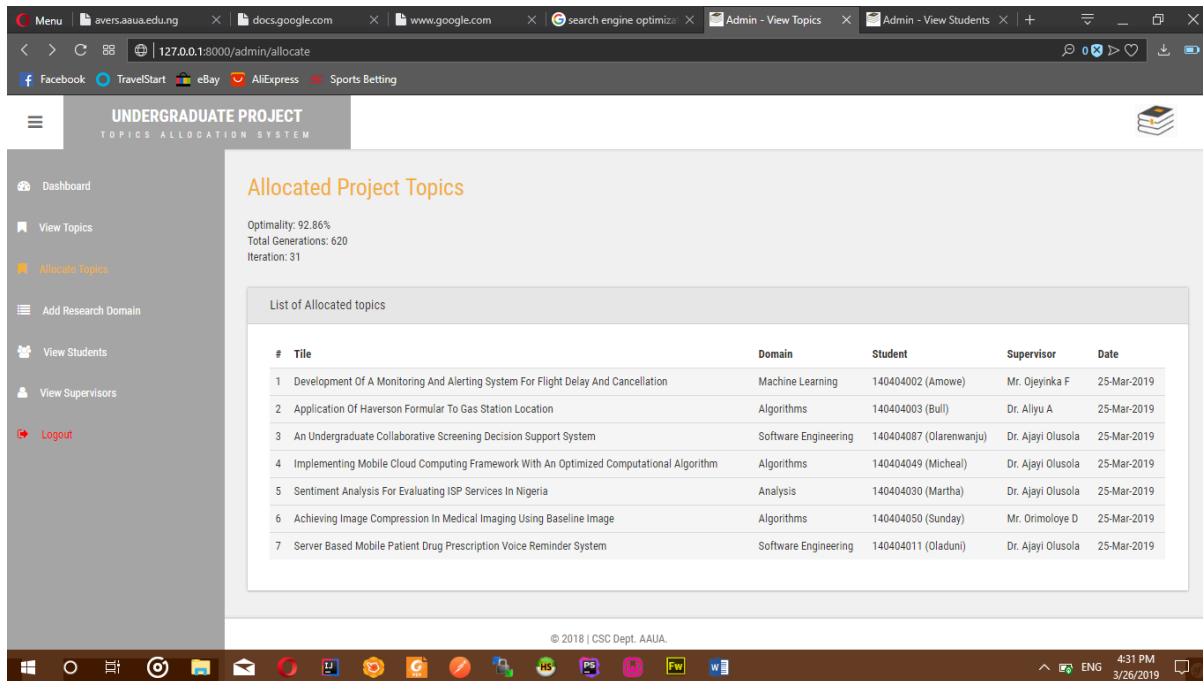
```

        return count($this→genes);
    }

    /**
     * @param DNA $dna
     * @return bool
     */
    public function equal(DNA $dna): bool {
        return $this→getGenes() ≡ $dna→getGenes()?true:false;
    }
}

```

APPENDIX III (Object Listing – Outputs)



Allocated Project Topics

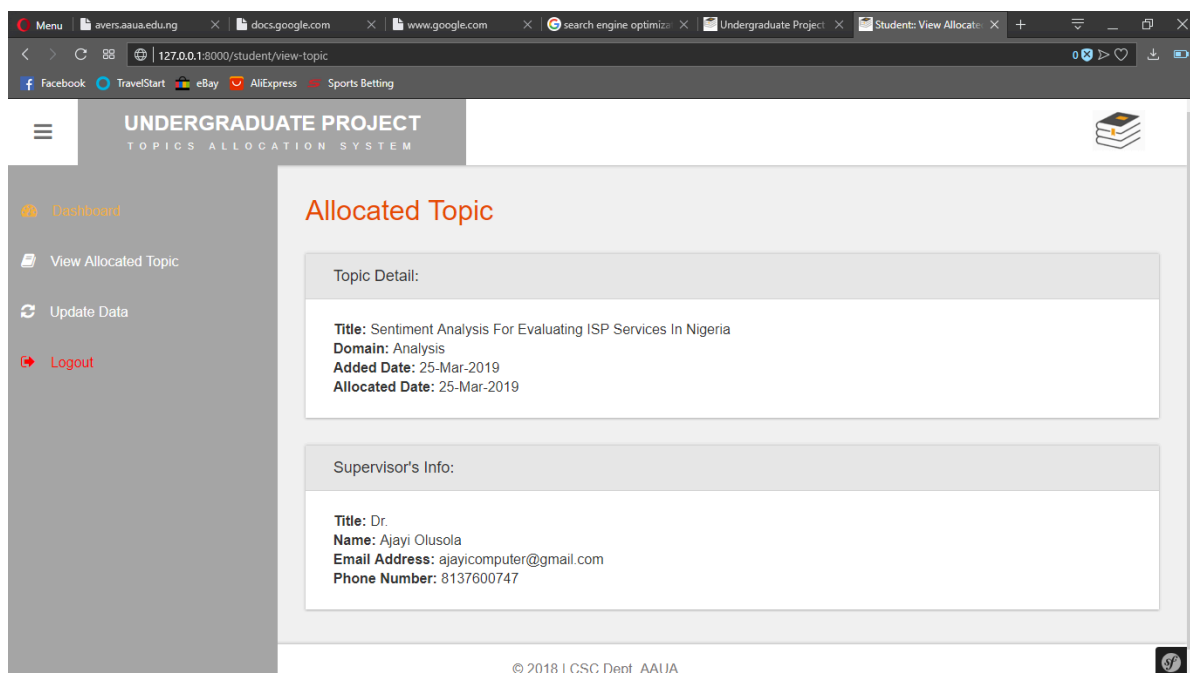
Optimality: 92.86%
Total Generations: 620
Iteration: 31

List of Allocated topics

#	Title	Domain	Student	Supervisor	Date
1	Development Of A Monitoring And Alerting System For Flight Delay And Cancellation	Machine Learning	140404002 (Amowe)	Mr. Ojeyinka F	25-Mar-2019
2	Application Of Haverson Formular To Gas Station Location	Algorithms	140404003 (Bull)	Dr. Aliyu A	25-Mar-2019
3	An Undergraduate Collaborative Screening Decision Support System	Software Engineering	140404087 (Olarenwaju)	Dr. Ajayi Olusola	25-Mar-2019
4	Implementing Mobile Cloud Computing Framework With An Optimized Computational Algorithm	Algorithms	140404049 (Micheal)	Dr. Ajayi Olusola	25-Mar-2019
5	Sentiment Analysis For Evaluating ISP Services In Nigeria	Analysis	140404030 (Martha)	Dr. Ajayi Olusola	25-Mar-2019
6	Achieving Image Compression In Medical Imaging Using Baseline Image	Algorithms	140404050 (Sunday)	Mr. Orimoloye D	25-Mar-2019
7	Server Based Mobile Patient Drug Prescription Voice Reminder System	Software Engineering	140404011 (Oladun)	Dr. Ajayi Olusola	25-Mar-2019

© 2018 | CSC Dept. AAUA.

Figure 6.1: Project topics allocation



Allocated Topic

Topic Detail:

Title: Sentiment Analysis For Evaluating ISP Services In Nigeria
Domain: Analysis
Added Date: 25-Mar-2019
Allocated Date: 25-Mar-2019

Supervisor's Info:

Title: Dr.
Name: Ajayi Olusola
Email Address: ajayicomputer@gmail.com
Phone Number: 8137600747

© 2018 | CSC Dept. AAUA.

Figure 6.2: A project topic successfully allocated to a particular student

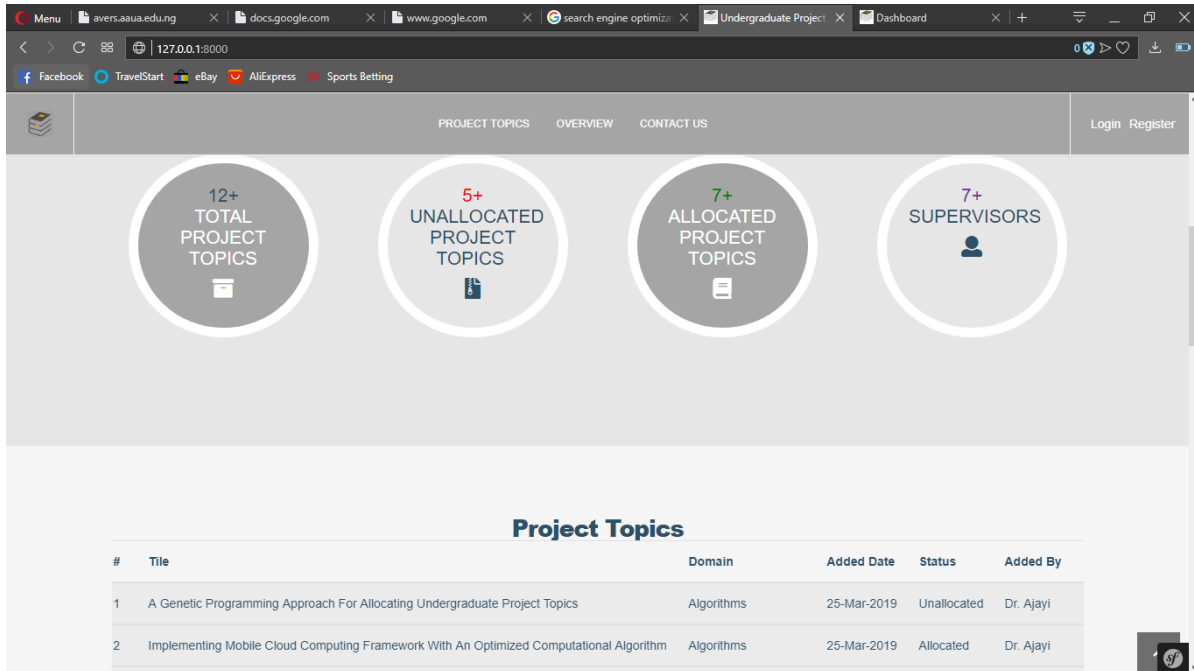


Figure 6.3: Landing page showing supervisors, allocated and unallocated project topics

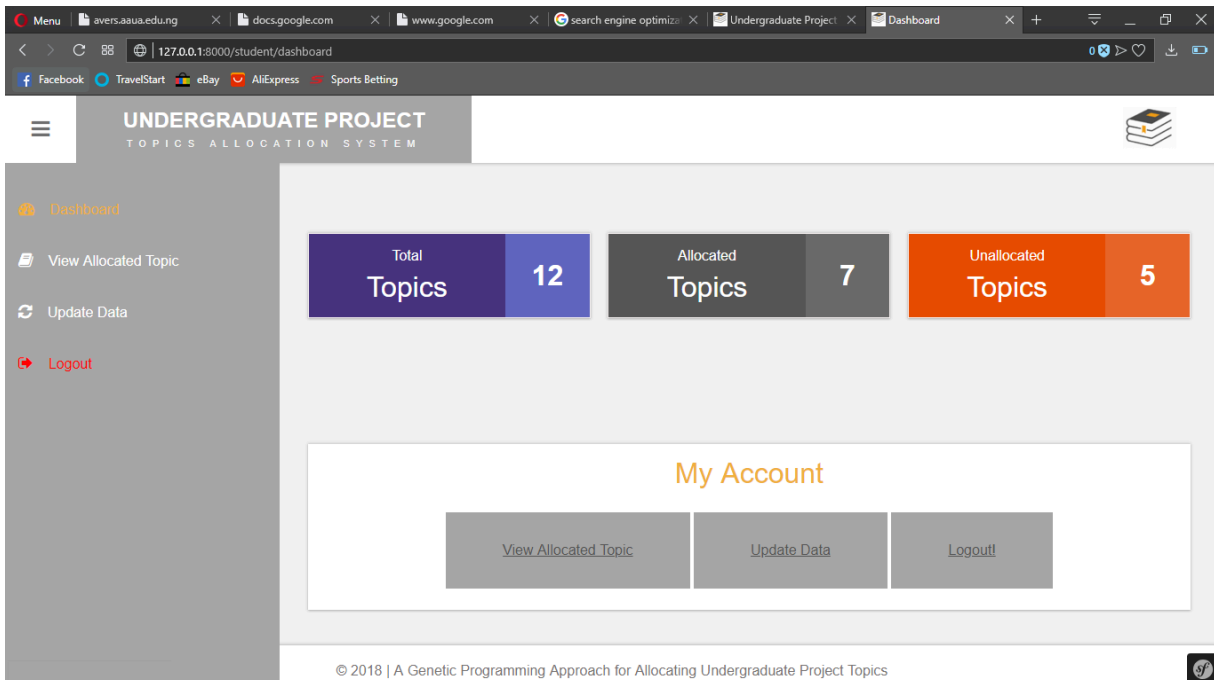


Figure 6.4: Student dashboard